

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Ngành: Trí Tuệ Nhân Tạo
Môn học: Tư duy tính toán cho Trí tuệ nhân tạo

Bài báo cáo: Tổng quan và Phân tích đồ án
AI English Learning RPG

Tên Nhóm: Nhóm Vibe coding Mixi
Giảng viên hướng dẫn: Ths. Nguyễn Lê Hoàng Dũng

Sinh viên thực hiện:

STT	Họ và Tên	Mã số sinh viên
1	Trần Phước Minh Khang	24122036
2	Nguyễn Việt Khoa	24122037
3	Huỳnh Tiến Mạnh	24122042
4	Hà Huy Nguyên	24122044
5	Dương Thiên Phú	24122047
6	Thái Hữu Phúc	24122049

ĐỒ ÁN HỌC KỲ II
CHƯƠNG TRÌNH CHÍNH QUY

Tp. Hồ Chí Minh, Tháng 3 Năm 2026

Mục lục

1 EXECUTIVE SUMMARY	3
1.1 Tổng quan cơ hội & Vấn đề giải quyết	3
1.2 Giải pháp & Giá trị cốt lõi	3
1.3 Công nghệ chủ chốt & Kiến trúc hệ thống	3
1.4 Kết quả kỳ vọng	4
2 PROJECT OVERVIEW	4
2.1 Bối cảnh dự án (Project Background)	4
2.2 Mục tiêu dự án (Project Objectives)	4
2.3 Phạm vi dự án (Project Scope)	5
2.4 Khung công nghệ sử dụng (Technology Stack)	5
3 PHÂN TÍCH VẤN ĐỀ (PROBLEM ANALYSIS)	6
3.1 Problem Statement (chuẩn hóa bài toán)	6
3.2 Các bên liên quan (Stakeholders)	7
3.3 Pain Points	9
3.4 Requirements	10
3.5 Constraints & Assumptions (Ràng buộc & Giả định)	10
3.5.1 Ràng buộc (Constraints)	10
3.5.2 Giả định (Assumptions)	10
3.6 Risk & Mitigation (Rủi ro & Biện pháp giảm thiểu)	11
3.7 Các Core Feature	12
4 COMPUTATIONAL THINKING FRAMEWORK	14
4.1 Decomposition (Phân rã bài toán)	14
4.2 Pattern Recognition (Nhận diện mẫu)	15
4.3 Abstraction (Trừu tượng hóa)	16
4.4 Algorithm Design (Thiết kế thuật toán)	16
5 SYSTEM DESIGN (Engineering Thinking)	16
5.1 Overall Architecture (Kiến trúc tổng thể)	17
5.2 Module Design	17
5.3 CT System Mapping (Traceability)	18
5.4 Data Flow (Luồng dữ liệu)	18
5.5 Use Case / Sequence Diagram	20
5.5.1 Use Cases	20
5.5.2 Sequence Diagram	23
5.6 Data Design & Modeling (Thiết kế Dữ liệu - Table Schema)	23
5.7 UI/UX Design & Interactive Prototype (Figma) DTP	24
5.7.1 Design Concept & Style Guide	24
5.7.2 High-Fidelity Mockups	24
5.7.3 Sơ đồ Figma	28
6 ALGORITHM DESIGN	28
6.1 Problem → Algorithm Mapping	28
6.1.1 Bài toán 1 — Đo lường năng lực từng từ vựng của người chơi	28
6.1.2 Bài toán 2 — Chọn từ vựng và điều chỉnh độ khó phù hợp với từng người chơi	29
6.1.3 Bài toán 3 — Sinh câu hỏi không gây trễ (latency = 0) khi vào trận	30
6.1.4 Bài toán 4 — Ràng buộc mô hình ngôn ngữ sinh output đúng format JSON	30

6.2	Input – Output Specification (tổng hợp)	32
6.3	Core Algorithms	32
6.3.1	Algorithm 1 — Knowledge Tracing (Mastery Score)	32
6.3.2	Algorithm 2 — DDA Pool Sampling (chọn từ yếu nhất)	33
6.3.3	Algorithm 3 — DDA Adaptive Prompt (điều chỉnh độ khó)	35
6.3.4	Algorithm 4 — Asynchronous Pre-generation Engine	37
6.3.5	Algorithm 5 — Structured Prompt Engineering & Double-parse	40
6.4	Optimization Strategy	43
6.4.1	Phần 1 — Cơ sở dữ liệu (SQLite)	43
6.4.2	Phần 2 — Thuật toán DDA	44
6.4.3	Phần 3 — Async Pre-generation Engine	44
6.4.4	Phần 4 — Prompt Engineering	45
6.4.5	Tổng kết các đánh đổi (Trade-offs)	46

1 EXECUTIVE SUMMARY

1.1 Tổng quan cơ hội & Vấn đề giải quyết

Phương pháp học tiếng Anh truyền thống thường dựa trên các bài tập rời rạc, dễ gây nhàm chán và thiếu môi trường thực hành thực tế. Dự án **AI English Learning RPG** ra đời nhằm thay đổi cách tiếp cận này bằng việc kết hợp giữa giáo dục và giải trí (Edutainment). Dự án hướng tới xây dựng một ứng dụng trò chơi di động nhập vai (Mobile RPG), nơi người học bắt buộc phải sử dụng tiếng Anh như một công cụ chính để sinh tồn, tương tác và hoàn thành các thử thách trong game thay vì làm bài tập một cách thụ động.

1.2 Giải pháp & Giá trị cốt lõi

Trọng tâm của giải pháp là việc ứng dụng Trí tuệ nhân tạo (AI) để cá nhân hóa lộ trình học tập của từng người thông qua một vòng lặp học tập tự động (Learning Loop):

Chơi (Play) → Sử dụng tiếng Anh (Use English) → Mặc lỗi (Make mistake) → AI phân tích lỗi (AI analyze) → Tạo nhiệm vụ phù hợp (Generate quest) → Áp dụng lại (Apply again)

Sản phẩm mang lại 4 giá trị giáo dục cốt lõi cho người dùng:

- **Tăng vốn từ vựng theo ngữ cảnh:** Từ vựng gắn liền với cốt truyện và tình huống trong game giúp ghi nhớ sâu.
- **Cải thiện ngữ pháp thực tế:** Học cách áp dụng cấu trúc câu thông qua giao tiếp thực tế với các nhân vật.
- **Tập phản xạ hội thoại:** Xây dựng tư duy phản xạ tiếng Anh tự nhiên.
- **Cá nhân hóa:** Hệ thống tự động nhận diện và tập trung cải thiện các điểm yếu riêng của từng người học.

1.3 Công nghệ chủ chốt & Kiến trúc hệ thống

Để hiện thực hóa trải nghiệm nhập vai thông minh, hệ thống loại bỏ các tính năng như sửa lỗi câu trực tiếp, nhận diện giọng nói (Speech-to-text) hay thuật toán phát âm để tập trung tối ưu hóa vào các công nghệ AI chạy cục bộ (Local) và hệ thống dữ liệu nhúng sau:

- **Trí tuệ nhân tạo (Generative AI & LLM):** Sử dụng cổng API của Ollama để vận hành mô hình ngôn ngữ lớn mã nguồn mở Qwen 2.5 (3B) trực tiếp trên thiết bị, giúp sinh lời thoại tự động cho NPC (*LLM dialogue generation*) và tự động thiết kế các nhiệm vụ học tập dựa trên khuôn mẫu có sẵn (*Quest generator LLM + template*).
- **Thuật toán cá nhân hóa:** Ứng dụng thuật toán lặp lại ngắt quãng (*Spaced repetition algorithm*) để ôn tập từ vựng, hệ thống phân tích hồ sơ lỗi của người chơi (*Player error profiling*) nhằm xác định điểm yếu, và bộ quy tắc điều chỉnh độ khó thích ứng (*Adaptive difficulty rules*) để tự động tăng/giảm thử thách trong game.
- **Nền tảng phát triển & Lưu trữ cục bộ:** Sử dụng Godot Engine (GDScript) làm nền tảng điều phối logic cốt lõi của game và tích hợp cơ sở dữ liệu nhúng SQLite ngay trên máy để lưu trữ tiến độ chơi, bộ sổ tay từ vựng (*notebook*) cùng nhật ký lỗi ngữ pháp của người dùng một cách bảo mật, độc lập và tối ưu hiệu năng.

1.4 Kết quả kỳ vọng

AI English Learning RPG kỳ vọng sẽ tạo ra một bước đột phá trong mảng ứng dụng học ngôn ngữ, biến quá trình học tập áp lực thành một trải nghiệm giải trí lôi cuốn. Dự án không chỉ chứng minh tính khả thi về mặt kỹ thuật trong việc tích hợp trực tiếp LLM chạy local (Ollama) vào công cụ phát triển game (Godot Engine) và hệ quản trị dữ liệu nhúng (SQLite) mà còn mang lại giá trị thực tiễn cao, giúp người học duy trì động lực và phát triển phản xạ tiếng Anh một cách tự nhiên nhất.

2 PROJECT OVERVIEW

2.1 Bối cảnh dự án (Project Background)

Việc duy trì động lực khi học một ngôn ngữ mới luôn là một thách thức lớn đối với hầu hết người học. Các phương pháp học truyền thống hoặc các ứng dụng hiện tại thường đi theo lối mòn: làm bài tập ngữ pháp điền vào chỗ trống, học từ vựng qua thẻ ghi nhớ (flashcard) một cách rời rạc, thiếu đi tính ngữ cảnh và môi trường thực hành.

Dự án **AI English Learning RPG** được phát triển nhằm xóa bỏ rào cản đó. Bằng cách lồng ghép việc học vào một trò chơi nhập vai di động (Mobile RPG), dự án biến tiếng Anh từ một "môn học áp lực" thành một "công cụ sinh tồn và khám phá". Người chơi buộc phải sử dụng ngôn ngữ để tương tác với thế giới trong game, từ đó hình thành phản xạ tự nhiên và duy trì được hứng thú học tập lâu dài.

2.2 Mục tiêu dự án (Project Objectives)

Về khía cạnh Học tập (Learning Objectives)

- **Tối ưu hóa từ vựng theo ngữ cảnh:** Giúp người học tiếp thu và ghi nhớ từ vựng thông qua cốt truyện, bối cảnh cốt truyện và các tình huống giao tiếp cụ thể trong game.
- **Ứng dụng ngữ pháp thực tế:** Tạo môi trường để người học vận dụng từ vựng khi tương tác, thay vì học vẹt lý thuyết.
- **Cá nhân hóa lộ trình:** Tự động nhận diện, phân tích và tập trung cải thiện các lỗ hổng kiến thức riêng biệt của từng người chơi.

Về khía cạnh Kỹ thuật (Technical Objectives)

- **Tích hợp Trí tuệ nhân tạo (LLM):** Ứng dụng thành công mô hình ngôn ngữ lớn để xử lý hội thoại động và tự động hóa việc thiết kế nhiệm vụ.
- **Xây dựng Vòng lặp học tập tự động (Learning Loop):** Thiết lập hệ thống khép kín từ lúc người chơi hành động, mắc lỗi, AI phân tích cho đến khi tạo ra thử thách mới để người chơi sửa sai.
- **Tối ưu hóa hiệu năng và lưu trữ local:** Xây dựng hệ thống vận hành mượt mà theo mô hình ứng dụng độc lập (Standalone Application), quản lý dữ liệu của người học một cách toàn vẹn thông qua SQLite mà không phụ thuộc vào kết nối Internet hay máy chủ đám mây bên thứ ba.

2.3 Phạm vi dự án (Project Scope)

Để đảm bảo dự án hoàn thành đúng tiến độ và tập trung vào các tính năng cốt lõi mang lại giá trị cao nhất, phạm vi công việc được quy định rõ ràng như sau:

Các tính năng nằm TRONG phạm vi phát triển (In-Scope)

- **LLM Dialogue Generation:** Hệ thống AI tạo lời thoại và phản hồi động cho các NPC dựa trên ngữ cảnh game và hành vi của người chơi.
- **Quest Generator (LLM + Template):** Tự động tạo các nhiệm vụ học thuật trong game bằng cách kết hợp sức mạnh của LLM và các khuôn mẫu định sẵn.
- **Player Error Profiling:** Trình phân tích ghi nhận và thiết lập hồ sơ các lỗi sai (từ vựng) mà người chơi gặp phải.
- **Adaptive Difficulty Rules:** Bộ quy tắc tự động điều chỉnh, tăng hoặc giảm độ khó của game dựa trên hồ sơ năng lực của người chơi.
- **Vocabulary Extraction & Spaced Repetition:** Hệ thống tự động trích xuất từ mới từ hội thoại và áp dụng thuật toán lặp lại ngắt quãng để hỗ trợ người chơi ôn tập.
- **Game Logic & Client Controller (Godot Engine / GDScript):** Xây dựng các module xử lý logic cốt lõi của trò chơi và điều phối luồng dữ liệu.
- **Local Database Management (SQLite):** Lưu trữ cục bộ và quản lý toàn vẹn tiến độ chơi, sổ tay từ vựng (notebook) và danh sách các lỗi ngữ pháp đã phạm phải.

Các tính năng nằm NGOÀI phạm vi phát triển (Out-of-Scope)

- **Pronunciation Heuristic:** Không phát triển công cụ tự động phát hiện và đánh giá lỗi phát âm sai.
- **Grammar Correction (LLM):** Không tích hợp tính năng LLM sửa lỗi câu trực tiếp ngay trong đoạn chat hội thoại.
- **Speech-to-text:** Không hỗ trợ tính năng nhập liệu hoặc giao tiếp bằng giọng nói (chỉ tập trung vào text-based nhập vai).

2.4 Khung công nghệ sử dụng (Technology Stack)

Để đảm bảo tính độc lập, tối ưu hóa hiệu năng xử lý local trên máy tính cá nhân (PC) và không phụ thuộc vào kết nối Internet hay chi phí dịch vụ đám mây bên thứ ba, dự án được xây dựng trên một kiến trúc Local-first hoàn chỉnh với các công nghệ sau:

- **Game Engine & Core Logic Platform: Godot Engine (GDScript)**
Được lựa chọn làm nền tảng phát triển tổng thể cho cả giao diện trò chơi (Client UI/UX) và bộ điều khiển logic trung tâm. GDScript đảm nhận nhiệm vụ xử lý trạng thái game, điều phối luồng dữ liệu, thực hiện các thuật toán lặp lại ngắt quãng (Spaced Repetition) và bộ quy tắc điều chỉnh độ khó thích ứng (Adaptive Difficulty Rules).
- **Local AI API Server & Models:**
 - **Ollama API:** Đóng vai trò là cổng API cục bộ (Local API Server) để quản lý và vận hành mô hình trí tuệ nhân tạo trực tiếp trên máy của người dùng. Godot sẽ giao tiếp với Ollama thông qua giao thức HTTP REST Client tiêu chuẩn (*HTTPRequest node*).

- **Qwen 2.5 (Phiên bản 3B - 3 Billion Parameters):** Mô hình ngôn ngữ lớn (LLM) mã nguồn mở chạy local, được tối ưu riêng cho các tác vụ ngôn ngữ và hội thoại. Model này chịu trách nhiệm sinh lời thoại tự động cho NPC (*LLM Dialogue Generation*) và phối hợp với các khuôn mẫu để tự động thiết kế nhiệm vụ học tập (*Quest Generator*).

- **Database Management System: SQLite**

Hệ quản trị cơ sở dữ liệu quan hệ nhúng (Embedded Database) nhỏ gọn, được tích hợp trực tiếp vào trong Project Godot thông qua *The godot-sqlite plugin*. Toàn bộ dữ liệu về tiến độ chơi, bộ sổ tay từ vựng cá nhân (Notebook), nhật ký lỗi sai và hồ sơ năng lực của người chơi sẽ được lưu trữ cục bộ một cách an toàn và truy xuất với tốc độ cao.

3 PHÂN TÍCH VẤN ĐỀ (PROBLEM ANALYSIS)

3.1 Problem Statement (chuẩn hóa bài toán)

Sinh viên thường mất động lực học tiếng Anh do ngữ pháp khô khan và từ vựng khó nhớ. Bài toán đặt ra là xây dựng một hệ thống RPG tích hợp AI để biến quá trình học thành trò chơi giải trí, tăng hứng thú và khả năng ghi nhớ thông qua tương tác cốt truyện.

1. Đầu vào (Input):

- Thông số ngữ cảnh hiện tại từ Game Engine truyền xuống: Cấp độ người chơi (Level bao gồm các mức độ: A1, A2, B1, B2)
- Hệ khuôn mẫu câu lệnh (System Prompt Template) được thiết lập sẵn để định hình cấu trúc dữ liệu đầu ra cho AI.

2. Đầu ra (Output):

- Một chuỗi dữ liệu văn bản đã được định dạng cấu trúc (JSON Object) chứa chính xác các trường thông tin: "question" (nội dung câu hỏi), "correct_answer" (đáp án đúng), và "distractors" (mảng gồm 3 đáp án gây nhiễu).

3. Ràng buộc (Constraints):

- *Ràng buộc kỹ thuật / Phần cứng:* Hệ thống phải chạy Offline 100% trên máy trạm (Local), không sử dụng API Internet bên ngoài, giới hạn dung lượng bộ nhớ RAM phân bổ cho AI nhỏ hơn hoặc bằng 8GB.

3.2 Các bên liên quan (Stakeholders)

Bảng 1: Bảng Stakeholder

Stakeholder	Vai trò	Nhu cầu/mong đợi	Ảnh hưởng	Mức độ quan trọng
Sinh viên	Người dùng cuối	Game mượt, từ vựng sát thực tế, cảm giác "lên trình" tiếng Anh qua các chỉ số (level, rank).	Cao	Cao
Nhóm phát triển	Đội ngũ thực thi	Quy trình code rõ ràng, tài liệu đầy đủ, không bị chồng chéo nhiệm vụ.	Cao	Cao
Giảng viên hướng dẫn	Người đánh giá	Áp dụng đúng đắn các quy tắc, công thức tư duy của môn học, báo cáo đúng tiến độ công việc.	Trung bình	Cao
Nhà cung cấp AI	Đối tác hạ tầng	API ổn định, chi phí thấp (hoặc miễn phí), phản hồi nhanh.	Thấp	Trung bình
Người yêu thích game/RPG	Người dùng mở rộng	Cốt truyện hấp dẫn, cơ chế chơi độc đáo, tương tác với NPC thú vị.	Thấp	Thấp

3.3 Pain Points

Bảng 2: Bảng tổng hợp các điểm đau (Pain points) từ khảo sát người dùng

STT	Pain point từ khảo sát	Mô tả chi tiết	Mức độ nghiêm trọng	% Người gặp phải	Tần suất
1	● Thiếu người sửa lỗi và giải thích cặn kẽ.	Người dùng không hài lòng khi các công cụ (Grammarly, Study4, sách ngữ pháp) "chỉ hiện dấu tích xanh/đỏ hoặc cung cấp đáp án đúng chứ không giải thích lý do đằng sau". Việc tự mày mò nguyên nhân tốn nhiều thời gian, ngại hỏi bạn bè nên lỗi sai dần trở thành thói quen không sửa được.	5	56,5%	Hàng ngày
2	● Áp lực đánh giá, "bí từ" khi ghép câu.	Có ý tưởng bằng tiếng Việt nhưng khi cần tự viết câu (nhắn tin, viết email, tham gia seminar Zoom) thì lúng túng, e ngại cách diễn đạt không đúng chuẩn dẫn đến việc tốn thời gian vô ích.	4	47,8%	Hàng tuần
3	● Học thụ động, thiếu môi trường thực hành.	Việc ghi chép flashcard (Anki, sổ tay) hay học app (Duingo) bị đánh giá là thụ động và rời rạc. Người được khảo sát cảm thấy các ví dụ quá máy móc, không có ngữ cảnh thực tế. Vì học chay và không có môi trường thực hành dẫn đến việc phản ứng chậm trong các tình huống thực tế.	4	43,5%	Hàng ngày
4	● Phương pháp học nhàm chán, rập khuôn.	Việc học bằng flashcard hay chọn đáp án A, B, C, D có sẵn trên app bị lặp đi lặp lại mà không có sự sáng tạo mới mẻ, gây ra chán nản và làm cho người học dễ bỏ cuộc. Người học cần một thể giới tương tác, có mục tiêu rõ ràng, sử dụng tiếng Anh làm công cụ để vượt qua thử thách.	3	39,1%	Hàng ngày

3.4 Requirements

- **Yêu cầu chức năng (Functional Requirements):**

- Hệ thống phải tự động nhận biết khi người chơi kích hoạt hội thoại với NPC để gửi lệnh sinh câu hỏi.
- Hệ thống phải kiểm tra tính đúng/sai của đáp án do người chơi chọn và cập nhật trình độ của Player.

- **Yêu cầu phi chức năng (Non-functional Requirements):**

- Mô hình AI Local có thời gian phản hồi thấp và hiệu suất cao, điều này cần tối ưu API để thời gian phản hồi ≤ 5 giây
- Độ chính xác của nội dung câu hỏi được AI sinh ra ổn định, tỷ lệ câu hỏi đúng ngữ pháp và cấu trúc $\geq 90\%$

3.5 Constraints & Assumptions (Ràng buộc & Giả định)

3.5.1 Ràng buộc (Constraints)

Đây là các giới hạn bắt buộc nhóm phát triển phải tuân thủ, không thể thay đổi do đặc thù của công nghệ và đề tài:

- **Ràng buộc Phần cứng & Tài nguyên (Hardware Constraints):** Hệ thống bắt buộc phải vận hành mượt mà trên các máy trạm cá nhân (PC) cấu hình phổ thông. Dung lượng bộ nhớ RAM phân bổ tối đa cho mô hình AI chạy cục bộ (Local LLM) phải thỏa mãn điều kiện $\leq 8\text{GB}$.
- **Ràng buộc Kết nối (Connectivity Constraints):** Ứng dụng phải hoạt động theo mô hình ngoại tuyến. Hệ thống phụ thuộc vào kết nối Internet hoặc gọi đến các API điện toán đám mây từ bên ngoài.
- **Ràng buộc về Phạm vi Tính năng (Scope Constraints):** Loại bỏ hoàn toàn các tính năng phức tạp bao gồm sửa lỗi câu trực tiếp từ LLM trong đoạn hội thoại (Grammar Correction), nhận diện giọng nói (Speech-to-text), và thuật toán đánh giá phát âm (Pronunciation Heuristic) để tập trung tài nguyên tối ưu cho trải nghiệm nhập vai dựa trên văn bản (Text-based RPG).
- **Ràng buộc về Định dạng Đầu ra (Output Formatting Constraints):** Phản hồi từ mô hình ngôn ngữ lớn chạy local phải trả về chính xác định dạng cấu trúc dữ liệu chuỗi văn bản đối tượng JSON (JSON Object) với các trường quy định rõ ràng bao gồm: "question", "correct_answer", và "distractors" để cấu phần Game Engine có thể bóc tách dữ liệu thành công.

3.5.2 Giả định (Assumptions)

Đây là các yếu tố được giả định là đúng để làm cơ sở và tiền đề thiết kế hệ thống kỹ thuật:

- **Giả định về năng lực Mô hình:** Giả định rằng mô hình mã nguồn mở Qwen 2.5 (phiên bản 3B Billion Parameters) khi vận hành cục bộ thông qua Ollama API có đủ khả năng hiểu ngữ cảnh trò chơi và sinh câu hỏi trắc nghiệm ổn định, đạt độ chính xác kiểm soát nội dung $\geq 90\%$ thông qua các kỹ thuật thiết lập khuôn mẫu câu lệnh (Prompt Engineering).

- **Giả định về hành vi Người dùng:** Giả định rằng đối tượng sinh viên (người dùng cuối) sở hữu trình độ năng lực tiếng Anh nền tảng nằm trong khung từ bậc A1 đến B2, và sẽ thực hiện tương tác làm bài một cách nghiêm túc để hệ thống xây dựng hồ sơ năng lực (Player Mastery Profile) chính xác.
- **Giả định về hiệu năng Godot-SQLite:** Giả định plugin `godot-sqlite` hoạt động ổn định trên Godot Engine, đảm bảo việc thực thi đọc/ghi dữ liệu lịch sử lỗi sai, tiến trình chơi và hàng đợi câu hỏi diễn ra bất đồng bộ mà không gây ra hiện tượng nghẽn luồng xử lý đồ họa và logic chính của trò chơi (Main Game Loop Thread).

3.6 Risk & Mitigation (Rủi ro & Biện pháp giảm thiểu)

Dưới đây là bảng tổng hợp phân tích các rủi ro kỹ thuật, kiến trúc vận hành cùng các giải pháp phòng ngừa, khắc phục tương ứng trong quá trình triển khai đề án:

Bảng 3: Bảng tổng hợp rủi ro và biện pháp giảm thiểu

STT	Rủi ro (Risk)	Mô tả chi tiết	Mức độ	Biện pháp giảm thiểu (Mitigation)
1	Độ trễ phản hồi AI cao	Mô hình LLM chạy local trên phần cứng yếu có thể mất từ 3–10 giây để sinh xong câu hỏi, gây ngắt quãng trải nghiệm chơi.	Cao	Ứng dụng mô hình Cỗ máy Sinh nội dung Bất đồng bộ (Asynchronous Pre-generation Engine) . Sử dụng luồng ngầm (Background Thread) để gọi Ollama sinh sẵn câu hỏi và nạp vào hàng đợi (Queue/Cache) ngay khi người chơi đang di chuyển trên bản đồ.
2	Lỗi định dạng cấu trúc JSON	Local LLM đôi khi sinh ra chuỗi văn bản tự do hoặc sai cấu trúc định dạng quy định, khiến Game Engine không thể phân tách (parse) và gây lỗi hệ thống.	Cao	Thiết lập hệ thống kiểm tra và nhận diện mẫu cấu trúc chuẩn bằng biểu thức chính quy Regex kết hợp cơ chế Try-Catch . Chỉ nạp vào Cache các chuỗi khớp 100% cấu trúc khuôn mẫu; nếu lỗi sẽ hủy và gửi lại request ngầm.
3	Hiện tượng ảo tưởng nội dung	AI sinh ra câu hỏi sai kiến thức ngữ pháp, từ vựng không thực tế hoặc các phương án gây nhiễu (<i>distractors</i>) vô tình trùng với đáp án đúng.	Trung bình	Chuẩn hóa cấu trúc Hệ khuôn mẫu câu lệnh (System Prompt Template) nghiêm ngặt. Áp dụng kỹ thuật cung cấp ngữ cảnh giới hạn phạm vi trích xuất dữ liệu từ vựng lấy từ SQLite tương ứng chặt chẽ với từng phân cấp Tier quái vật.
4	Tràn bộ nhớ RAM hệ thống	Máy trạm của người dùng không đủ tài nguyên phần cứng hoặc tiến trình Ollama chiếm dụng vượt mức cho phép dẫn đến đứng máy, giật lag.	Trung bình	Lựa chọn sử dụng phiên bản mô hình ngôn ngữ tối ưu đã qua lượng tử hóa (Quantized) là Qwen 2.5 (3B) nhằm giảm thiểu dung lượng phần cứng.
5	Mất mát/sai lệch dữ liệu học tập	SQLite gặp lỗi đồng bộ ghi dữ liệu khi người chơi tắt ứng dụng đột ngột, dẫn đến tính toán sai lệch điểm thông thạo (<i>Mastery Score</i>) ở các vòng lặp sau.	Thấp	Thiết kế module quản lý cơ sở dữ liệu tuân thủ cơ chế Transaction (Commit/Rollback) của SQLite. Thiết lập tiến trình tự động lưu dữ liệu (Auto-save) ngay sau khi hoàn thành phân giải kết quả mỗi lượt đánh (Combat Resolution).

3.7 Các Core Feature

Từ các ý vụn:

1. Hệ thống phản hồi AI tức thời (Real-time AI Feedback Engine)
2. Tạo câu hỏi dựa trên trình độ người chơi

3. Chấm điểm, giải thích và sửa lỗi
4. Tăng giảm độ khó
5. Xác định điểm yếu từ vựng/ ngữ pháp
6. Hệ thống tổng hợp từ vựng ôn tập cho người chơi
7. Phát hiện từ mới/ từ làm sai

Ta suy ra được các Core Feature dưới đây:

1. **Feature 1: Hệ thống Sinh nội dung Thích ứng (Adaptive Content Generation & DDA)** (Gồm ý 2 và 4)
 - **Mô tả:** Thay vì sử dụng ngân hàng câu hỏi tĩnh, hệ thống tích hợp thuật toán *Điều chỉnh độ khó động* (*Dynamic Difficulty Adjustment - DDA*) kết hợp với AI Prompting. Dựa trên mức độ thành thạo hiện tại của người chơi, Godot sẽ điều chỉnh prompt gửi tới Ollama để sinh ra các câu hỏi trắc nghiệm hoàn toàn phù hợp với trình độ người chơi theo thời gian thực.
 - **Giải quyết Pain Point:** Xóa bỏ sự nhàm chán của việc lặp lại câu hỏi cố định; giữ người chơi ở trạng thái "Flow" (không quá dễ gây chán, không quá khó gây nản).
2. **Feature 2: Hệ thống Đánh giá & Phản hồi AI Tức thời (Real-time Evaluation & Explainable AI)** (Gồm ý 1 và 3)
 - **Mô tả:** Ngay sau mỗi lượt tương tác/chiến đấu, thay vì chỉ trả về kết quả Đúng/Sai (Binary evaluation), hệ thống sử dụng năng lực của LLM để thực hiện *Phản hồi có thể giải thích* (*Explainable AI Feedback*). Cụ thể, AI sẽ chấm điểm, chỉ ra lỗi sai cốt lõi và cung cấp lời giải thích ngắn gọn, dễ hiểu, đặt chuẩn xác vào bối cảnh của câu hỏi vừa làm.
 - **Giải quyết Pain Point:** Giúp người học hiểu sâu bản chất vấn đề thay vì học vẹt (nhớ đáp án A, B, C). Cung cấp trải nghiệm như có một "Gia sư 1-1" đồng hành ngay trong trận chiến.
3. **Feature 3: Hệ thống Theo dõi Kiến thức & Cá nhân hóa Ôn tập (Knowledge Tracing)** (Gồm ý 5, 6 và 7)
 - **Mô tả:** Mọi tương tác của người chơi được phân tích và lưu trữ vào SQLite để tạo thành một "Hồ sơ năng lực" (Player Mastery Profile). Hệ thống áp dụng mô hình *Theo dõi kiến thức* (*Knowledge Tracing*) để tự động nhận diện các từ vựng mới, định lượng điểm yếu từ vựng (dựa trên tần suất trả lời sai) và tổng hợp để người chơi tự ôn tập.
 - **Giải quyết Pain Point:** Khắc phục triệt để "đường cong quên lãng" (Ebbinghaus forgetting curve). Tối ưu hóa thời gian học bằng cách cá nhân hóa 100% lộ trình: chỉ học những gì mình chưa biết hoặc hay làm sai.
4. **Feature 4: Hệ thống Sinh nội dung Theo ngữ cảnh Game (Context-Aware Thematic Prompting)**
 - **Mô tả:** Thay vì sinh các câu hỏi trắc nghiệm ngữ pháp/từ vựng với nội dung ngẫu nhiên, hệ thống sẽ chèn thông số về môi trường (Biome) hoặc loại quái vật (Enemy Type) hiện tại của người chơi vào *AI Prompt*. Ví dụ: Nếu người chơi đang ở "Khu rừng Lửa" (Fire Cave) và cần học từ "Vulnerable", AI (Ollama) sẽ sinh ra một câu hỏi trắc nghiệm có nội dung về hiệp sĩ, ngọn lửa, rồng hoặc sự sinh tồn.

- **Giải quyết Pain Point:** Người học thường cảm thấy chán nản vì phải làm các câu hỏi ngữ pháp/từ vựng khô khan, rời rạc, không liên quan đến nhau. Tính năng này giúp gắn kết chặt chẽ (Immersive) trải nghiệm học tập vào cốt truyện và bối cảnh game, tăng sự hứng thú.

5. Feature 5: Hệ thống Phân cấp Năng lực theo Tuyến Quái vật (Enemy-bound Hierarchical Competency Gating)

- **Mô tả:** Trong phạm vi một bản đồ thế giới mở (Chapter 1), hệ thống dữ liệu từ vựng trong SQLite được phân mảnh (Data Partitioning) và gắn kết trực tiếp với từng loại quái vật theo cấu trúc Cây phụ thuộc (Dependency Tree). Cụ thể, kiến thức được mở khóa tuần tự dựa trên tiến trình đánh quái của người chơi:
 - **Tier 1 (Ví dụ: Slime/Cấp cơ bản):** AI chỉ truy vấn và sinh câu hỏi trắc nghiệm liên quan đến từ vựng phổ thông (A1) và thi hiện tại đơn.
 - **Tier 2 (Ví dụ: Goblin/Cấp trung bình):** Người chơi bắt buộc phải đánh bại đủ số lượng quái Tier 1 để hệ thống mở khóa dữ liệu Tier 2. Lúc này, khi chạm trán quái Tier 2, độ khó của câu hỏi sẽ tự động nâng lên vùng từ vựng chuyên sâu hơn.
 - **Boss Encounter (Bài kiểm tra tổng hợp):** Trận đánh Boss cuối map sẽ đóng vai trò như bài test tổng hợp (Summative Assessment), trộn lẫn toàn bộ kiến thức người chơi đã học từ các loại quái trước đó.
- **Giải quyết Pain Point:** Người học tiếng Anh thường bị "ngợp" trước danh sách từ vựng khổng lồ, thiếu một lộ trình học có cấu trúc đi từ nền tảng đến nâng cao. Đồng thời, trong các game học tập, người chơi có xu hướng né tránh các bài tập khó nếu hệ thống cho phép chọn tự do.
- **Giải pháp:** Biến lộ trình học tập (Learning Path) thành lộ trình khám phá bản đồ game. Cơ chế chặn cấp (Gating) bằng sức mạnh quái vật ép người chơi phải thành thạo kiến thức cơ bản thì mới có cơ hội "sống sót" để đi sâu hơn vào bản đồ. Nó tạo ra cảm giác tiến bộ rõ rệt (Sense of Progression) mà không cần phải chuyển đổi qua lại giữa các màn chơi khác nhau.

6. Feature 6 (Tính năng Kỹ thuật): Cổ máy Sinh nội dung Bất đồng bộ (Asynchronous Pre-generation Engine)

- **Mô tả:** Hệ thống sở hữu một module chạy ngầm (background thread) trên Godot. Thấu hiểu việc chạy mô hình ngôn ngữ Local (Ollama) trên máy tính cá nhân sẽ tốn từ 3-10 giây để trả kết quả, module này sẽ *âm thầm truy vấn dữ liệu từ SQLite và gửi request cho AI ngay từ lúc người chơi đang đi bộ trên bản đồ*. Khi người chơi thực sự chạm trán quái vật, câu hỏi và đáp án đã được chuẩn bị sẵn trong bộ nhớ đệm (Cache), giúp trận đấu bắt đầu ngay lập tức với độ trễ (latency) bằng 0.
- **Giải quyết Pain Point:** Trải nghiệm (UX) tệ hại của các ứng dụng tích hợp AI Local là bắt người dùng phải chờ AI một thời gian lâu. Tính năng này giải quyết triệt để rào cản phần cứng yếu, giữ cho nhịp độ game (gameplay flow) diễn ra mượt mà không bị gián đoạn.

4 COMPUTATIONAL THINKING FRAMEWORK

4.1 Decomposition (Phân rã bài toán)

Thay vì xử lý toàn bộ khối lượng kiến thức tiếng Anh và hệ thống game trong một luồng nguyên khối, hệ thống phân rã bài toán trên 3 khía cạnh:

1. Functional Decomposition (Phân rã theo cụm chức năng logic):

- **Progression & Gating Logic (Logic kiểm soát tiến trình):** Quản lý điều kiện mở khóa các phân vùng kiến thức khó hơn (Tier 2, Boss) dựa trên số lượng quái Tier 1 đã bị người chơi tiêu diệt.
- **Asynchronous AI Generation Logic (Logic sinh nội dung bất đồng bộ):** Chịu trách nhiệm ngàm truy vấn SQLite và gọi Ollama API để chuẩn bị sẵn các câu hỏi trắc nghiệm (A, B, C, D) vào hàng đợi (Queue), loại bỏ hoàn toàn độ trễ (latency) khi load trận.
- **Combat Resolution Logic (Logic phân giải chiến đấu):** Đánh giá đáp án trắc nghiệm người chơi chọn so với JSON của AI, từ đó quy đổi thành các lệnh trừ HP quái/người chơi và hiển thị giải thích.

2. Task Decomposition (Phân rã luồng Sinh & Đánh giá câu hỏi trắc nghiệm):

Để một lượt đánh (turn) diễn ra chính xác mà không gây quá tải tài nguyên tính toán, quá trình này được chia thành các bước tuần tự:

- **Bước 1 (Chuẩn bị bối cảnh):** Godot trích xuất *weak_vocab* từ SQLite và *biome_context* ghép thành Prompt.
- **Bước 2 (Sinh dữ liệu):** Ollama nhận Prompt, tạo ra 1 câu hỏi, 1 đáp án đúng, 3 đáp án nhiễu (*distractors*) và 1 lời giải thích, trả về dưới dạng chuỗi JSON.
- **Bước 3 (Bóc tách & Hiển thị):** Godot parse JSON, xáo trộn (*shuffle*) ngẫu nhiên vị trí 4 nút A, B, C, D hiển thị lên UI.
- **Bước 4 (Đánh giá & Cập nhật):** Xác thực thao tác click của người dùng, thực thi hiệu ứng game và lưu tiến độ xuống DB.

3. Data Decomposition (Phân rã cấu trúc dữ liệu):

Dữ liệu được chia nhỏ (*Data Partitioning*) để tối ưu truy vấn trong SQLite:

- **Enemy-bound Vocabulary (Dữ liệu tĩnh):** Kho từ vựng không gom chung một bảng lớn mà được gắn chặt với từng loại quái vật (Slime mang tập từ vựng A1, Goblin mang tập từ vựng A2).
- **Knowledge Tracing (Dữ liệu động):** Hồ sơ học tập phân rã thành các tham số nhỏ theo từng từ: *encounter_count* (số lần gặp), *correct_count* (số lần đúng) để hệ thống định lượng được Điểm thông thạo (*Mastery Score*).

4.2 Pattern Recognition (Nhận diện mẫu)

Hệ thống sử dụng khả năng Nhận diện mẫu trên hai khía cạnh quản lý dữ liệu:

- **Nhận diện mẫu lỗi học tập (Knowledge Tracing Pattern):** Dựa vào lịch sử chọn đáp án trắc nghiệm lưu trong SQLite, thuật toán quét và nhận diện các "Mẫu điểm yếu" (*Pattern of weaknesses*). Những từ vựng/ngữ pháp có tỷ lệ chọn sai lặp lại liên tục sẽ tạo thành một pattern kích hoạt cơ chế Lặp lại ngắt quãng (*Spaced Repetition*), ép AI ưu tiên đưa các từ này vào Prompt sinh đề cho các trận chiến tiếp theo.

- **Nhận diện mẫu Cấu trúc Dữ liệu AI (JSON Parsing Pattern):** Vì Local LLM đôi khi sinh ra các văn bản có định dạng bất thường, hệ thống Godot sử dụng Regex và cơ chế Try-Catch để nhận diện "Mẫu cấu trúc chuẩn". Chỉ khi phản hồi của AI khớp chính xác với mẫu { "question": "", "options": [], "correct": "", "explain": "" } thì hệ thống mới chấp nhận nạp vào Cache, nếu không sẽ loại bỏ để tránh crash game.

4.3 Abstraction (Trừu tượng hóa)

Để AI (một mô hình ngôn ngữ) giao tiếp mượt mà với Game Engine (môi trường toán học/đồ họa), quá trình trừu tượng hóa tiến hành lọc bỏ các nhiễu đồ họa và số hóa ngôn ngữ:

- **Trừu tượng hóa Bối cảnh Game (Context Abstraction):** LLM không cần biết tọa độ X/Y của nhân vật hay hiệu ứng âm thanh. Game trừu tượng hóa trạng thái hiện tại thành bộ tham số đơn giản truyền vào Prompt: *biome* (Hang động/Rừng), *enemy_type* (Slime/Rồng), và *vocabulary_target*.
- **Trừu tượng hóa Đánh giá Ngôn ngữ thành Chỉ số RPG:** Sự phức tạp của việc học ngoại ngữ được trừu tượng hóa và đóng gói thành cơ chế Game cơ bản: Trả lời đúng đáp án = *Hit_Damage* (Sát thương lên quái) + Tăng *Mastery_Score*; Trả lời sai = *Take_Damage* (Bị quái đánh) + Gọi hàm *Show_Explorable_AI*.

4.4 Algorithm Design (Thiết kế thuật toán)

Từ các thành phần trên, nhóm thiết kế 2 luồng thuật toán cốt lõi để đảm bảo hiệu năng và cá nhân hóa:

- **Thuật toán Tải ngầm Bất đồng bộ (Asynchronous Pre-fetching Algorithm):** Chạy luồng ngầm (*Background Thread*) liên tục kiểm tra trạng thái hàng đợi câu hỏi (*Queue*). Nếu số lượng câu hỏi trong Queue < 3, thuật toán tự động bốc từ vừng điểm yếu từ SQLite, gọi API Ollama sinh sẵn JSON và nạp vào hàng đợi ngay lúc người chơi đang đi bộ khám phá Map. (Đảm bảo Zero-latency - không có độ trễ khi bước vào Combat).
- **Thuật toán Điều chỉnh Độ khó Động (Dynamic Difficulty Adjustment - DDA):** Thuật toán tính toán Điểm thông thạo dựa trên tỷ lệ phản hồi chính xác:

$$\text{Mastery Score} = \frac{\text{correct_count}}{\text{encounter_count}}$$

- Nếu Mastery Score < 0.5: Tần suất từ vừng này xuất hiện tăng lên, prompt AI tạo câu hỏi trắc nghiệm có từ vựng ngữ cảnh đơn giản hơn để người chơi lấy lại căn bản.
- Nếu Mastery Score > 0.8: Thuật toán mở khóa trích xuất các từ vựng thuộc cấp độ cao hơn (Tier tiếp theo) để đẩy vào Prompt, nâng dần thách thức cho người chơi.

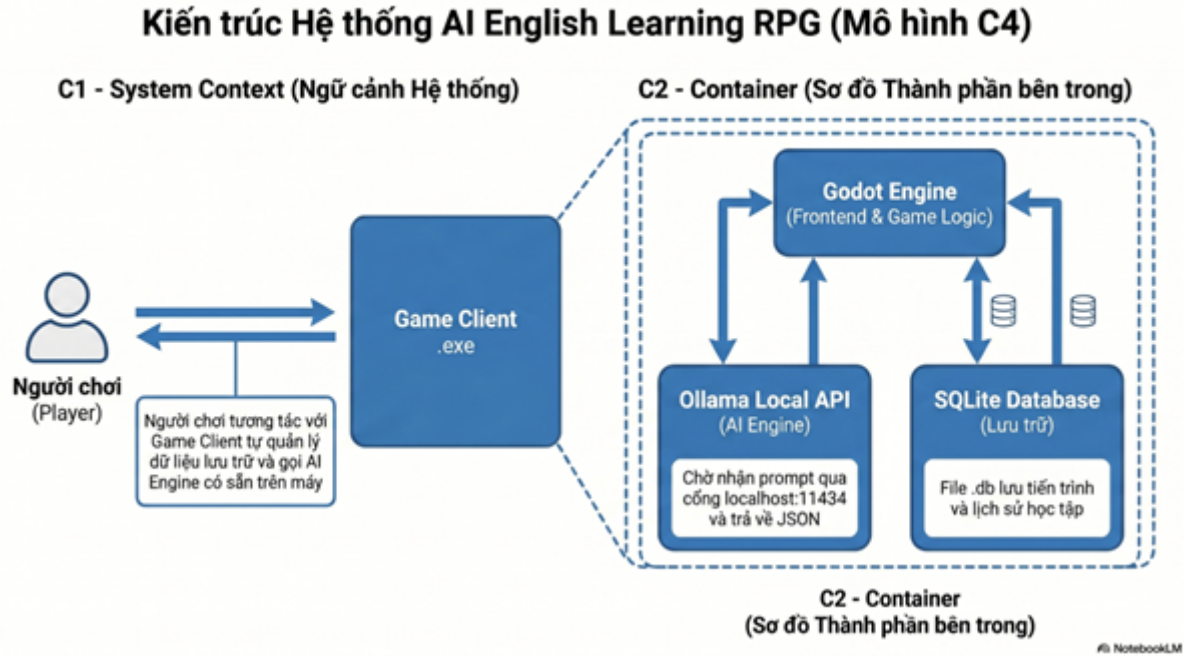
5 SYSTEM DESIGN (Engineering Thinking)

Hệ thống được thiết kế dựa trên nguyên lý **Decomposition** (Phân rã hệ thống) và **Abstraction** (Trừu tượng hóa dữ liệu), chia nhỏ một bài toán học tập ngôn ngữ phức tạp thành các module phần mềm độc lập nhưng giao tiếp chặt chẽ với nhau trên môi trường cục bộ (Local).

5.1 Overall Architecture (Kiến trúc tổng thể)

Vì game chạy 100% local, kiến trúc hệ thống sẽ là một ứng dụng Desktop nguyên khối (Monolithic Desktop App) giao tiếp với các dịch vụ cục bộ.

Sơ đồ C4 (gồm C1: context và C2: container):



Hình 1: Sơ đồ C4

5.2 Module Design

Hệ thống được chia thành 4 module chính:

1. UI/UX Module (Godot):

- *Scene Manager*: Quản lý luồng hiển thị giữa Bản đồ thế giới và Màn hình chiến đấu.
- *Real-time Evaluation UI*: Hiển thị câu hỏi, nút đáp án và giao diện xuất hiện của NPC AI kèm lời giải thích.

2. Core Game Logic (Godot):

- *Asynchronous Pre-generation Engine*: Chạy ngầm (background thread). Âm thầm truy vấn SQLite và gọi API Ollama khi người chơi đang di chuyển trên bản đồ, lưu kết quả vào Cache để loại bỏ hoàn toàn độ trễ khi vào trận đấu.
- *Hierarchical Competency Gating*: Giám sát tiến độ tiêu diệt quái vật để mở khóa các phân vùng từ vựng khó hơn (Tier 1, Tier 2).

3. AI Integration Module (Godot + Ollama):

- *Context-Aware Prompt Builder*: Dựa vào thông số loại quái vật và môi trường xung quanh (Biome) để chèn vào System Prompt, gửi tới AI để sinh đề bài khớp ngữ cảnh.
- *JSON Parser*: Bóc tách dữ liệu phản hồi từ AI lấy cấu trúc câu hỏi, đáp án, và phần giải thích (Explainable AI).

4. Data Tracking Module (SQLite):

- *Knowledge Tracing System*: Lưu trữ tỷ lệ trả lời đúng/sai của người chơi trên từng từ vựng cụ thể.

5.3 CT System Mapping (Traceability)

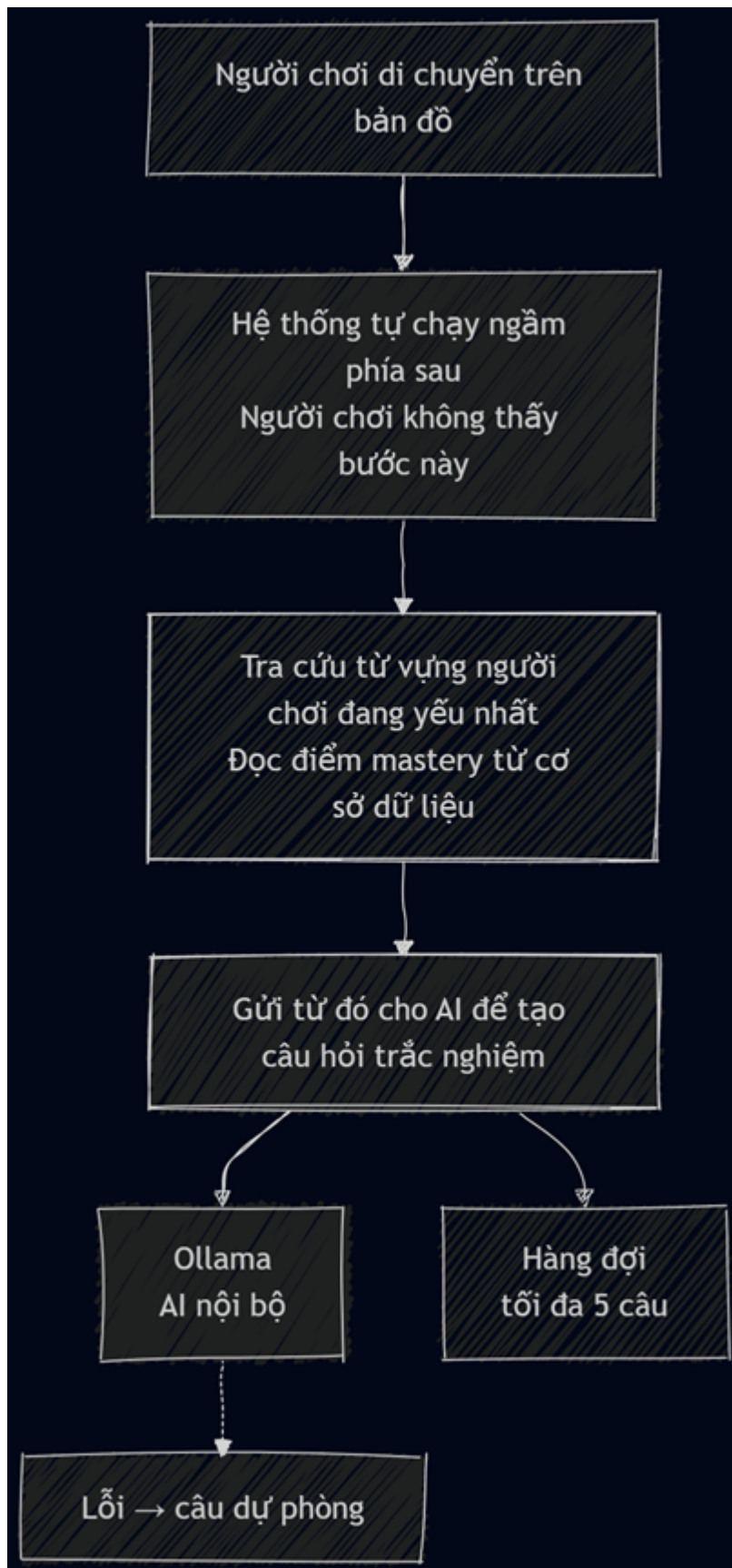
Bảng ánh xạ dưới đây thể hiện cách các trụ cột của Tư duy tính toán định hình nên kiến trúc hệ thống:

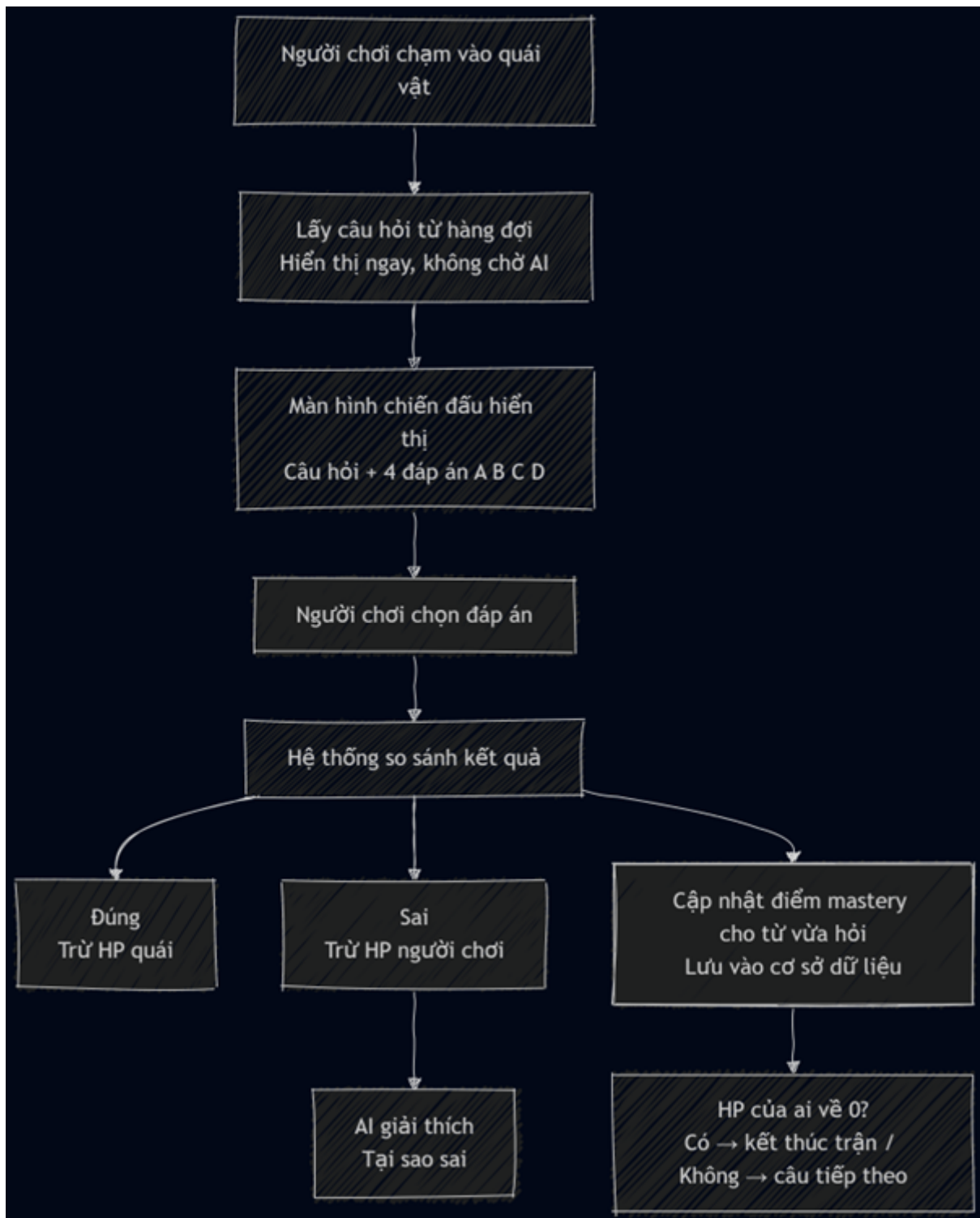
Bảng 4: Ma trận liên kết Trụ cột Tư duy tính toán và Thành phần Hệ thống

CT Pillar (Trụ cột TDDT)	Mạch Logic (Business Logic)	System Component (Thành phần hệ thống)	Liên kết Core Feature
Decomposition (Phân rã bài toán)	Chia nhỏ toàn bộ dữ liệu từ vựng khổng lồ thành các mảnh dữ liệu nhỏ, gắn chặt với từng phân cấp quái vật và khu vực cụ thể trên Map.	Hierarchical Gating Controller (Godot) & Data Partitioning (SQLite)	Feature 5
Pattern Recognition (Nhận diện mẫu)	Theo dõi và nhận diện các mẫu lỗi sai (<i>pattern of mistakes</i>) lặp lại thường xuyên của người chơi trong lúc làm bài để định lượng điểm yếu.	Knowledge Tracing System (SQLite Database)	Feature 3
Abstraction (Trừu tượng hóa)	Lọc bỏ các chi tiết thừa, biến đổi bối cảnh phức tạp của game (biome, loại quái, level) thành các tham số cốt lõi để ghép thành Prompt giao tiếp với AI.	Context-Aware Prompt Builder (Godot)	Feature 4
Algorithm Design (Thiết kế thuật toán)	Xây dựng luồng xử lý bất đồng bộ (chạy ngầm) để sinh đề/chấm điểm và thuật toán điều chỉnh độ khó động (DDA) dựa trên trình độ người chơi.	Async Pre-generation Engine (Godot) + Ollama Local API	Feature 1, 2 & 6

5.4 Data Flow (Luồng dữ liệu)

Sơ đồ luồng dữ liệu (Data Flow Diagram - DFD) dạng khối hình học (Flowchart diagram):





5.5 Use Case / Sequence Diagram

5.5.1 Use Cases

Use Case 1: Bắt đầu hoặc tiếp tục hành trình

Tên Use Case: Khởi tạo / Tải tiến trình game

Mục tiêu: Cho phép người chơi bắt đầu một hành trình học từ vựng mới hoặc tiếp tục từ lần chơi trước.

Tác nhân: Người chơi, Cơ sở dữ liệu SQLite

Điều kiện tiên quyết: Người chơi đã mở file game thành công trên máy tính.

Hậu điều kiện: Màn hình bản đồ hiển thị, nhân vật sẵn sàng di chuyển.

Các bước thực hiện:

1. Hệ thống hiển thị màn hình chính với hai lựa chọn: **New Game** và **Continue**.
2. Nếu người chơi chọn **New Game**: hệ thống tạo hồ sơ mới (HP đầy, điểm mastery về mức ban đầu) và đưa người chơi vào Chapter 1.
3. Nếu người chơi chọn **Continue**: hệ thống đọc file **save.db** để khôi phục vị trí trên bản đồ, HP, và lịch sử học từ vựng.
4. Hệ thống tải tài nguyên game và đặt nhân vật đúng vị trí đã lưu.

Kịch bản phụ: Nếu file lưu bị mất hoặc hỏng, hệ thống ấn nút **Continue**, hiển thị thông báo “Không tìm thấy dữ liệu lưu” và yêu cầu người chơi chọn **New Game**.

Use Case 2: Khám phá bản đồ

Tên Use Case: Di chuyển và khám phá thế giới game

Mục tiêu: Người chơi di chuyển tự do trên bản đồ, trong khi hệ thống tự động chuẩn bị sẵn câu hỏi cho trận chiến tiếp theo.

Tác nhân: Người chơi, Ollama (AI chạy trên máy người chơi)

Điều kiện tiên quyết: Người chơi đang ở màn hình bản đồ thế giới.

Hậu điều kiện: Luôn có sẵn tối đa 5 câu hỏi trong bộ nhớ đệm, sẵn sàng cho trận chiến bất cứ lúc nào.

Các bước thực hiện:

1. Người chơi dùng phím WASD để di chuyển nhân vật trên bản đồ.
2. Song song với việc người chơi di chuyển, hệ thống âm thanh chạy ngầm phía sau.
3. Hệ thống tra cứu danh sách từ vựng mà người chơi đang yếu nhất trong tier hiện tại.
4. Hệ thống gửi yêu cầu đến Ollama để tạo câu hỏi trắc nghiệm phù hợp với trình độ.
5. Câu hỏi được tạo xong sẽ được lưu vào hàng đợi bộ nhớ, không hiển thị ngay cho người chơi.

Kịch bản phụ: Nếu Ollama phản hồi lỗi hoặc quá chậm, hệ thống tự động điền câu hỏi dự phòng có sẵn vào hàng đợi để game không bị gián đoạn. Sau 2 giây, hệ thống tự thử kết nối lại Ollama mà không cần người chơi can thiệp.

Use Case 3: Chạm trán quái vật

Tên Use Case: Tiếp cận và kích hoạt trận chiến

Mục tiêu: Kiểm tra xem người chơi có đủ điều kiện để thách đấu loại quái vật hiện tại không, sau đó chuyển sang màn hình chiến đấu.

Tác nhân: Người chơi

Điều kiện tiên quyết: Người chơi đang di chuyển trên bản đồ và tiếp cận một quái vật.

Hậu điều kiện: Màn hình chuyển sang giao diện chiến đấu, hoặc người chơi được yêu cầu luyện tập thêm với quái cấp thấp hơn.

Các bước thực hiện:

1. Người chơi di chuyển nhân vật đến gần và chạm vào quái vật.

2. Hệ thống đọc cấp độ (tier) của quái vật — ví dụ: Slime thuộc Tier 1, Goblin thuộc Tier 2.
3. Hệ thống kiểm tra tiến trình của người chơi xem đã đáp ứng điều kiện mở khóa quái vật này chưa.
4. Nếu đủ điều kiện, màn hình chuyển cảnh sang giao diện chiến đấu.

Kịch bản phụ: Nếu người chơi chưa đủ điều kiện (ví dụ: cố tấn công Goblin khi chưa chinh phục đủ Slime), hệ thống hiển thị hộp thoại: “Vốn từ vựng của bạn chưa đủ để vào khu vực này. Hãy luyện tập thêm với quái vật ở khu vực trước!” và giữ nguyên màn hình bản đồ.

Use Case 4: Chiến đấu trắc nghiệm

Tên Use Case: Trả lời câu hỏi để chiến đấu

Mục tiêu: Người chơi trả lời câu hỏi trắc nghiệm từ vựng tiếng Anh để tấn công quái vật, đồng thời nhận giải thích tức thì từ AI khi trả lời sai.

Tác nhân: Người chơi

Điều kiện tiên quyết: Người chơi đã vào màn hình chiến đấu và hàng đợi câu hỏi đã có sẵn dữ liệu.

Hậu điều kiện: Kết thúc trận chiến, điểm mastery của từ vựng liên quan được cập nhật vào cơ sở dữ liệu.

Các bước thực hiện:

1. Hệ thống hiển thị câu hỏi trắc nghiệm kèm 4 đáp án A, B, C, D.
2. Người chơi đọc câu hỏi và chọn một đáp án bằng cách click.
3. Hệ thống đối chiếu đáp án của người chơi với đáp án đúng.
4. Nếu đúng: nhân vật tấn công quái vật, trừ HP quái.
5. Nếu sai: quái vật phản công, trừ HP người chơi. Hệ thống hiển thị bảng giải thích của AI nêu rõ tại sao đáp án đó sai.
6. Hệ thống ghi lại kết quả vào hồ sơ từ vựng, cập nhật điểm mastery của từ vừa xuất hiện.
7. Vòng lặp tiếp tục với câu hỏi mới cho đến khi HP của quái vật hoặc của người chơi về 0.

Kịch bản phụ: Nếu câu hỏi lấy từ hàng đợi bị lỗi định dạng, hệ thống tự động bỏ qua câu đó và lấy câu tiếp theo ngay lập tức, người chơi không nhận ra sự gián đoạn. Nếu hàng đợi hoàn toàn trống, hệ thống sử dụng câu hỏi dự phòng có sẵn (`word_id = -1`) và không ghi kết quả vào cơ sở dữ liệu để tránh dữ liệu rác.

Use Case 5: Xem sổ tay từ vựng

Tên Use Case: Tra cứu và ôn tập từ vựng đã học

Mục tiêu: Người chơi xem lại toàn bộ từ vựng đã gặp trong game, biết từ nào mình đang yếu để chủ động ôn tập.

Tác nhân: Người chơi

Điều kiện tiên quyết: Người chơi đang ở màn hình bản đồ, không trong trạng thái chiến đấu.

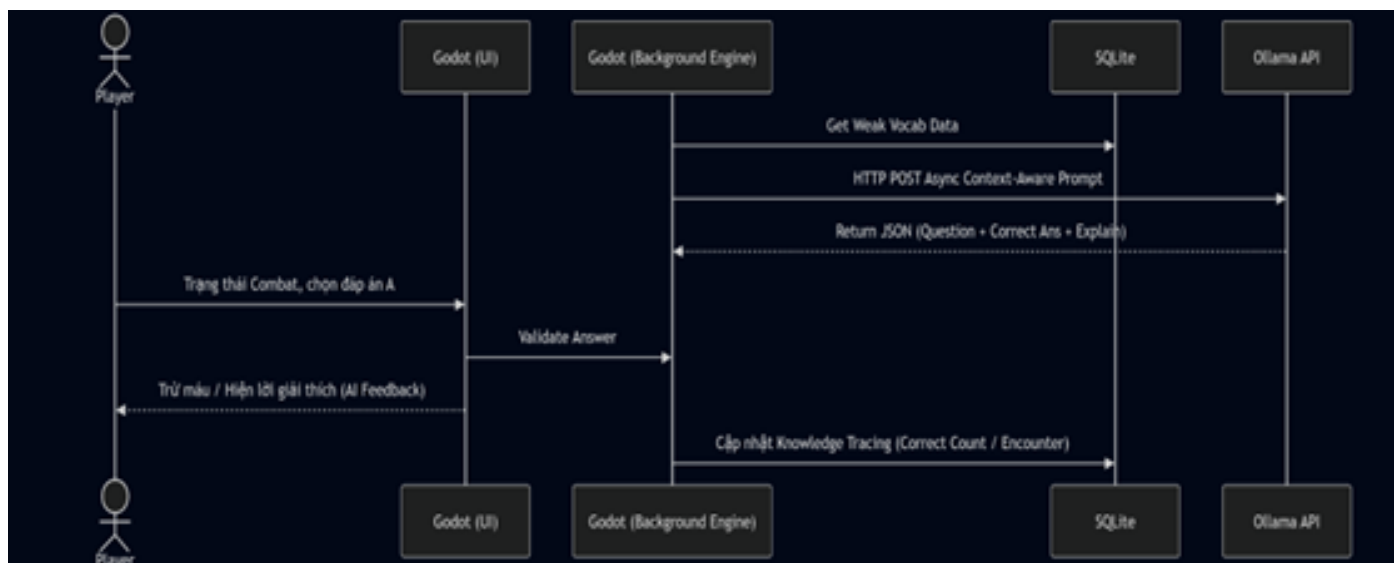
Hậu điều kiện: Người chơi đóng sổ tay và tiếp tục điều khiển nhân vật bình thường.

Các bước thực hiện:

1. Người chơi nhấn phím E hoặc click vào biểu tượng quyển sách trên màn hình.
2. Hệ thống truy vấn dữ liệu học tập của người chơi từ cơ sở dữ liệu.
3. Giao diện sổ tay hiển thị danh sách từ vựng đã gặp, phân loại bằng màu sắc: đỏ là từ hay sai, xanh lá là từ đã thành thạo.
4. Người chơi click vào một từ bất kỳ để xem nghĩa, ví dụ sử dụng và gợi ý ôn tập.
5. Người chơi click nút Thoát để đóng sổ tay và quay lại game.

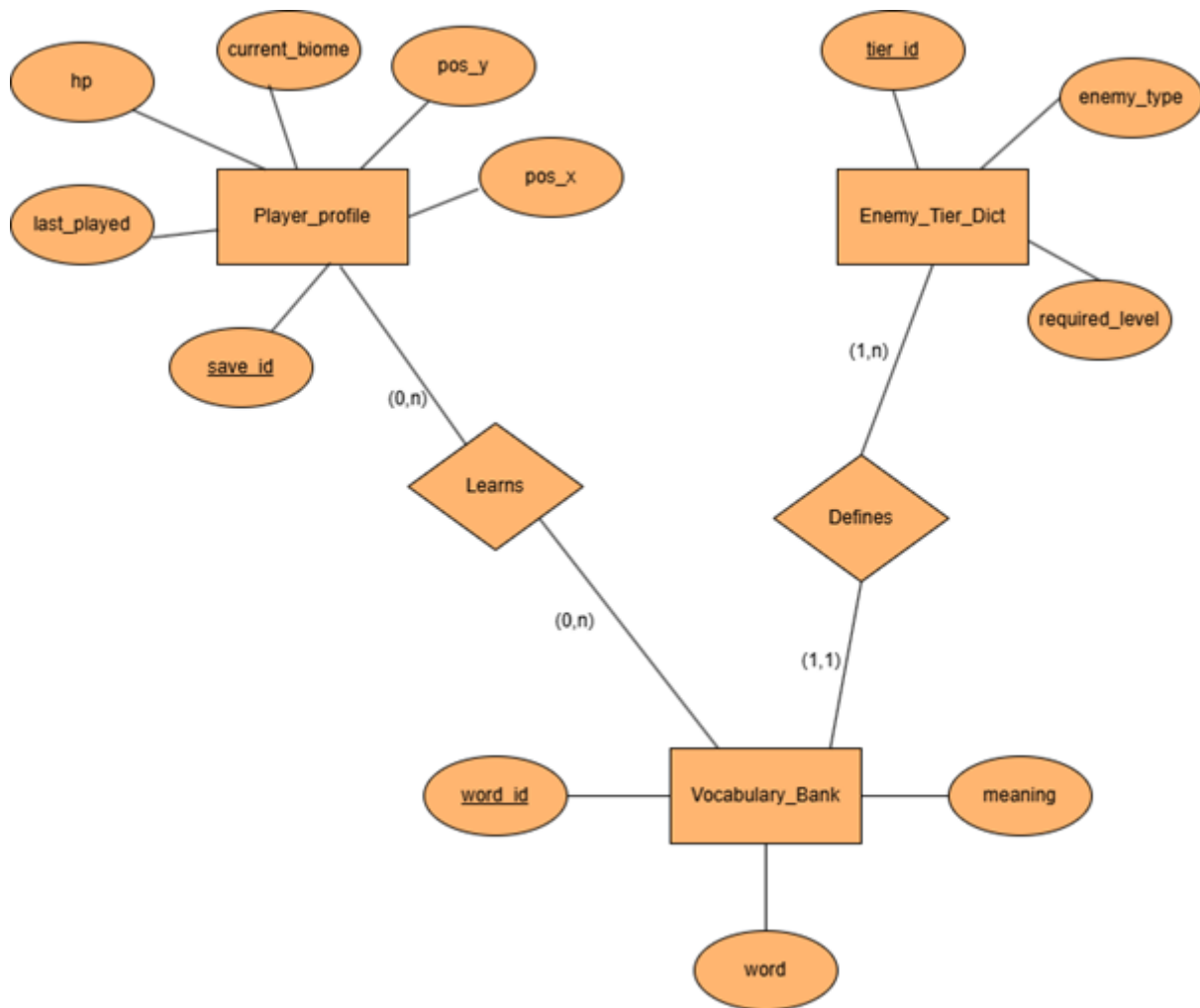
Kịch bản phụ: Nếu người chơi chưa gặp từ vựng nào (ví dụ mới bắt đầu game), sổ tay hiển thị thông báo “Bạn chưa gặp từ vựng nào. Hãy bắt đầu khám phá!” thay vì danh sách trống.

5.5.2 Sequence Diagram



5.6 Data Design & Modeling (Thiết kế Dữ liệu - Table Schema)

Table schema/ERD:



5.7 UI/UX Design & Interactive Prototype (Figma) DTP

5.7.1 Design Concept & Style Guide

5.7.2 High-Fidelity Mockups

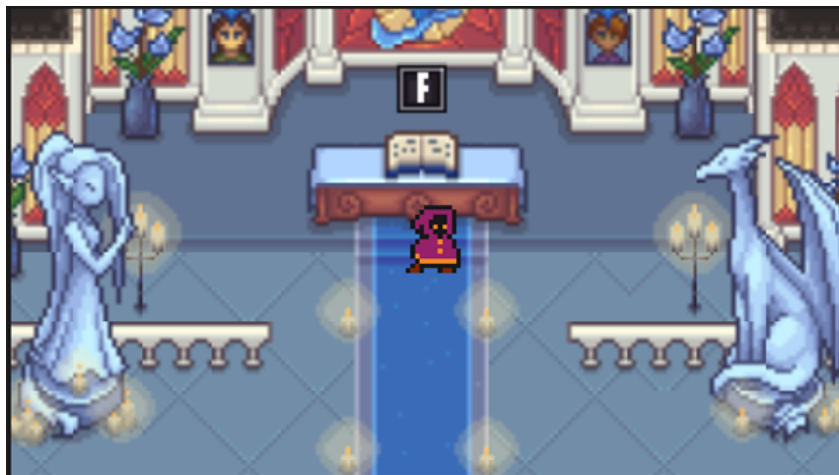
Dựa trên luồng trải nghiệm thực tế của người chơi, các giao diện được thiết kế và sắp xếp theo trình tự sau:

- **Màn hình chính:** Điểm chạm đầu tiên của người chơi. Gồm logo tên game ở vị trí trung tâm với phông nền là ảnh sơ bộ của hòn đảo đầu tiên góc nhìn từ trên xuống. Bao gồm các nút điều hướng cơ bản: *New game (bắt đầu)*, *continue (chơi tiếp)*, *options (cài đặt)* và *quit game (thoát)*.



- **Màn hình khám phá:** Giao diện góc nhìn từ trên xuống hoặc nhìn ngang tùy theo khu vực. Trực quan hóa nhân vật chính và môi trường xung quanh (nhà cửa, cây cối, NPC, quái vật). Giao diện này được thiết kế tối giản, loại bỏ các nút bấm không cần thiết để người chơi tập trung vào việc di chuyển và tìm kiếm mục tiêu tương tác, cụ thể có 3 khu vực như sau:

- Khu vực nhà thờ nơi người chơi tỉnh dậy và bắt đầu trò chơi.



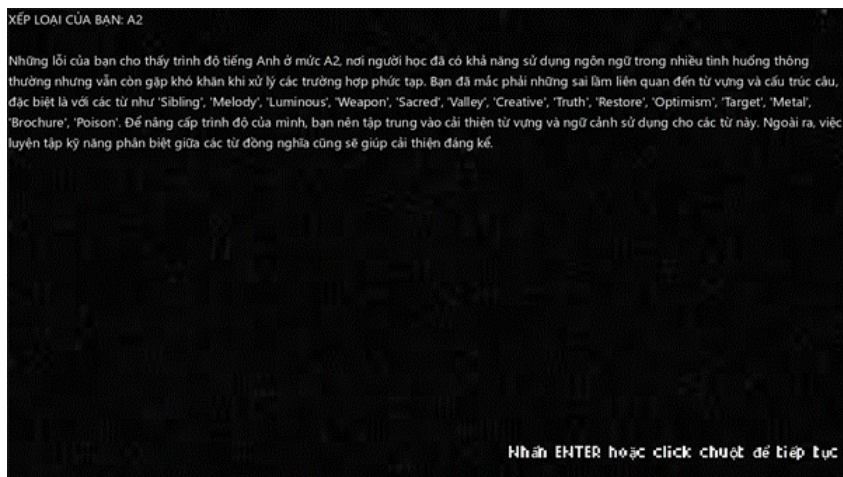
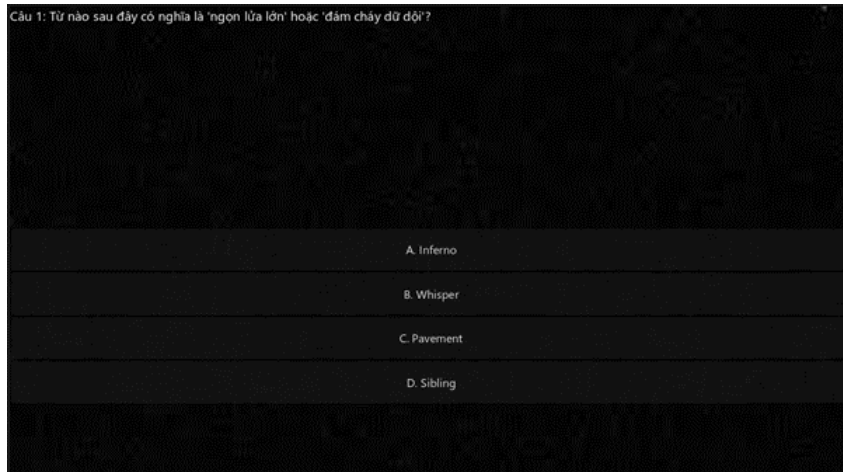
- Khu vực bên trong quyển sách nơi người chơi sẽ gặp npc hướng dẫn và tiếp nhận bài kiểm tra.



- Khu vực bên ngoài nhà thờ nơi đầy rẫy quái vật và người chơi phải tự mình khám phá.



- **Màn hình kiểm tra:** sau khi tương tác với npc hướng dẫn và màn hình kết quả sau một lần kiểm tra (cho biết trình độ người chơi):



- **Màn hình chiến đấu:** Đây là màn hình có khá nhiều chi tiết, ở góc trên bên trái và phải lần lượt là số máu của người chơi và quái vật. Ở giữa phần máu sẽ là nơi hiển thị câu hỏi, cùng với các lựa chọn nằm ở phần giữa dưới cùng. Nếu trả lời sai thì sẽ xuất hiện pop up AI giải thích nằm giữa người chơi và quái vật.



- Cuối cùng, chính là **màn hình kết quả trận đấu**: Màn hình này sẽ có 2 giao diện tùy theo kết quả là thua hay thắng.





5.7.3 Sơ đồ Figma

<https://www.figma.com/design/HnMILOSGWcdUy0RT2fc36h/Untitled?node-id=0-1&t=i2gctwBXCzkZ>

6 ALGORITHM DESIGN

6.1 Problem → Algorithm Mapping

6.1.1 Bài toán 1 — Đo lường năng lực từng từ vựng của người chơi

Mục	Nội dung
Problem	Hệ thống cần biết người chơi đang thành thạo từ nào và yếu từ nào, để ưu tiên ôn tập từ chưa vững.
Input	<code>encounter_count</code> (số lần gặp từ), <code>correct_count</code> (số lần trả lời đúng), lưu trong bảng <code>Player_Vocab_Mastery</code>
Output	<code>mastery_score</code> $\in [0.0, 1.0]$ — điểm thành thạo của từng từ
Constraints	Từ chưa gặp lần nào không được gây lỗi chia-cho-0; điểm phải phản ánh tỉ lệ đúng thực tế, không bị overfit khi gặp ít lần
Algorithm type	Scoring / Statistical Tracking
Thuật toán chọn	Knowledge Tracing — Mastery Score
Công thức	$\text{mastery_score} = \frac{\text{correct_count}}{\text{encounter_count} + 1}$
Lý do chọn	Đơn giản, $O(1)$, phù hợp với hệ thống realtime game. Mẫu số +1 giải quyết cả edge case từ chưa gặp lần giảm nhẹ điểm để tránh overconfidence.
Module	<code>DatabaseManager.update_after_combat()</code> → <code>Player_Vocab_Mastery</code>

6.1.2 Bài toán 2 — Chọn từ vựng và điều chỉnh độ khó phù hợp với từng người chơi

Mục	Nội dung
Problem	Mỗi lần vào trận, hệ thống phải chọn từ nào để hỏi và đặt câu hỏi ở mức khó nào để vừa thử thách, vừa không làm người chơi nản.
Input	<code>tier_id</code> (tier quái hiện tại), <code>mastery_score</code> từng từ trong tier, <code>avg_mastery</code> toàn tier
Output	Một từ mục tiêu (<code>word</code> , <code>meaning</code> , <code>word_id</code>) + chuỗi <code>difficulty_instruction</code> gửi vào prompt AI
Constraints	Phải ưu tiên từ yếu; không được ket mãi một nhóm từ khi tất cả mastery bằng nhau; không tăng độ khó khi người chơi chưa sẵn sàng
Algorithm type	Priority Sampling + Rule-based Heuristic
Thuật toán chọn	DDA — Dynamic Difficulty Adjustment (2 bước)
Lý do chọn	<code>ORDER BY RANDOM()</code> thuần không đảm bảo chọn từ yếu; pool-sampling 2 bước kết hợp được cả ưu tiên (top-15 yếu nhất) lẫn ngẫu nhiên (tránh ket). Rule-based heuristic trên mastery đủ nhẹ cho local game, không cần ML.
Module	<code>DatabaseManager.get_weakest_vocab()</code> + <code>AIManager._resolve_difficulty_instruction()</code>

Hai tầng của DDA:

- **Tầng 1 — Chọn từ (Pool Sampling):** SQL lấy 15 từ có `mastery_score` thấp nhất trong tier (`ORDER BY mastery_score ASC, RANDOM() LIMIT 15`), GDScript chọn ngẫu nhiên 1 trong pool đó.
- **Tầng 2 — Chỉnh độ khó (Adaptive Prompt):** Đọc `mastery_score` của từ được chọn và `avg_mastery` toàn tier, ánh xạ sang chỉ thị ngôn ngữ nhét vào prompt Ollama.

Điều kiện	Chỉ thị gửi cho AI
<code>encounter_count = 0</code>	Câu hỏi nhận diện nghĩa trực tiếp, đáp án sai rõ ràng
<code>mastery < 0.3</code>	Củng cố nghĩa cơ bản, không tăng độ khó
<code>0.3 ≤ mastery < 0.6</code>	Câu trung bình, đồng/trái nghĩa
<code>0.3 ≤ mastery < 0.6 + tier_is_advanced</code>	Điền vào chỗ trống, ngữ cảnh phong phú
<code>mastery ≥ 0.6</code>	Sắc thái nghĩa (nuance), đáp án sai tinh tế

6.1.3 Bài toán 3 — Sinh câu hỏi không gây trễ (latency = 0) khi vào trận

Mục	Nội dung
Problem	Ollama chạy local tốn 3–10 giây mỗi request. Nếu sinh câu hỏi theo yêu cầu, người chơi phải chờ trước mỗi trận — UX tệ.
Input	<code>current_tier_id</code> , <code>is_generating</code> flag, <code>question_queues[tier_id]</code> , <code>MAX_QUEUE_SIZE = 5</code>
Output	Câu hỏi MCQ JSON lấy tức thì từ RAM (<code>question_queues[tier_id].pop_front()</code>)
Constraints	Ollama chỉ xử lý 1 request tại 1 thời điểm; cần đảm bảo queue không rỗng khi vào trận; phải có fallback khi AI lỗi
Algorithm type	Producer-Consumer Queue + Priority Scheduling
Thuật toán chọn	Asynchronous Pre-generation Engine với Multi-tier Queue
Lý do chọn	Queue riêng theo tier thay vì queue chung giải quyết được vấn đề latency khi chuyển tier; priority scheduling (ưu tiên nạp tier hiện tại) đảm bảo không bao giờ trống khi cần.
Module	<code>AIManager.check_and_fill_all_queues()</code> , <code>AIManager.get_question()</code>

Chiến lược ưu tiên nạp queue:

- **Ưu tiên 1** — Nạp tier hiện tại người chơi đang dùng (đảm bảo không trống khi vào trận).
- **Ưu tiên 2** — Nạp ngậm các tier còn thiếu theo thứ tự trong `MANAGED_TIERS`.
- **Fallback** — Nếu queue rỗng: trả câu hardcoded với `word_id = -1`; `DatabaseManager` nhận diện để bỏ qua ghi mastery, tránh dữ liệu rác.
- **Auto-retry** — Khi AI lỗi (`response_code ≠ 200` hoặc JSON hỏng): chờ 2 giây rồi tự gọi lại, không cần can thiệp thủ công.

6.1.4 Bài toán 4 — Ràng buộc mô hình ngôn ngữ sinh output đúng format JSON

Mục	Nội dung
Problem	Qwen2.5-3B chạy local có độ tin cậy thấp hơn cloud model — dễ sinh text tự do thay vì JSON chuẩn, gây crash khi parse.
Input	<code>word</code> , <code>meaning</code> , <code>difficulty_instruction</code> , JSON schema mong muốn
Output	Object JSON hợp lệ: <code>{ question, A, B, C, D, correct_answer, explanation }</code>
Constraints	Model có thể thêm text ngoài JSON; có thể bọc trong markdown fence; response là 2 lớp JSON lồng nhau (Ollama wrapper + nội dung AI)
Algorithm type	Structured Output Engineering + Defensive Parsing
Thuật toán chọn	4-part Structured Prompt + Double-parse + Validation
Lý do chọn	Kết hợp ràng buộc tại cấp model (format: json của Ollama) và cấp prompt (khai báo schema tường minh, cấm text ngoài JSON) tạo 2 lớp bảo vệ. Double-parse và validation trước khi append vào queue đảm bảo không crash game dù AI có trả output hỏng.
Module	<code>AIManager._build_prompt()</code> , <code>AIManager._on_request_completed()</code>

Cấu trúc prompt 4 phần:

1. **Role** — Neo AI vào bối cảnh: "Bạn là Elaria, hệ thống tạo thử thách tiếng Anh cho game RPG."
2. **Target word** — Từ bắt buộc phải xuất hiện trong câu hỏi: **word + meaning**.
3. **Difficulty instruction** — Output của `_resolve_difficulty_instruction()` từ DDA.
4. **Output rules** — Khai báo JSON schema, cấm thêm bất kỳ text nào ngoài JSON.

Double-parse pipeline: Ollama response

- Parse lớp ngoài → { message: { content: '...' } }
- Parse lớp trong → { question, A, B, C, D, correct_answer, explanation }
- Validate: has('question') AND has('correct_answer') AND word_id ≠ -1
 - Append vào question_queues[tier_id] ✓
 - Discard nếu validate fail ✗ (không crash)

6.2 Input – Output Specification (tổng hợp)

#	Bài toán	Input	Output
1	Knowledge Tracing	encounter_count, correct_count	mastery_score $\in [0.0, 1.0]$
2	DDA — Chọn từ	tier_id, bảng Player_Vocab_Mastery	word_id, word, meaning
2	DDA — Chính độ khó	mastery_score, avg_mastery, encounter_count	difficulty_instruction (string)
3	Async Pre-gen	tier_id, Ollama endpoint	MCQ JSON trong RAM queue
4	Prompt Engineering	word, meaning, difficulty_instruction	question, A, B, C, D, correct_answer, explanation

6.3 Core Algorithms

6.3.1 Algorithm 1 — Knowledge Tracing (Mastery Score)

Ý tưởng: Mỗi từ vựng cần được gắn một điểm số phản ánh mức độ thành thạo thực sự của người chơi. Thay vì chỉ lưu tỉ lệ đúng đơn thuần ($\text{correct}/\text{encounter}$), mẫu số được cộng thêm 1 để xử lý đồng thời hai edge case: từ chưa gặp lần nào (tránh chia cho 0) và từ mới gặp ít lần nhưng luôn đúng (giảm nhẹ điểm để không overfit). Mỗi lượt chiến đấu đều cập nhật tức thì vào SQLite qua UPSERT, đảm bảo DDA luôn đọc được trạng thái mới nhất.

Pseudocode:

```
1 FUNCTION update_after_combat(save_id, word_id, is_correct):
2     IF word_id == -1:
3         RETURN -- bỏ qua câu fallback, không ghi dữ liệu rác
4
5     record = DB.query(
6         "SELECT encounter_count, correct_count
7         FROM Player_Vocab_Mastery
8         WHERE save_id = ? AND word_id = ?"
9     )
10
11     IF record EXISTS:
12         new_encounter = record.encounter_count + 1
13         new_correct = record.correct_count + (1 IF is_correct ELSE 0)
14         DB.execute(
15             "UPDATE Player_Vocab_Mastery
16             SET encounter_count = ?, correct_count = ?
17             WHERE save_id = ? AND word_id = ?"
18         )
19     ELSE:
20         DB.execute(
21             "INSERT INTO Player_Vocab_Mastery
22             (save_id, word_id, encounter_count, correct_count)
23             VALUES (?, ?, 1, ?)"
24         )
25
26     mastery_score = new_correct / (new_encounter + 1)
27     RETURN mastery_score
28
```

```

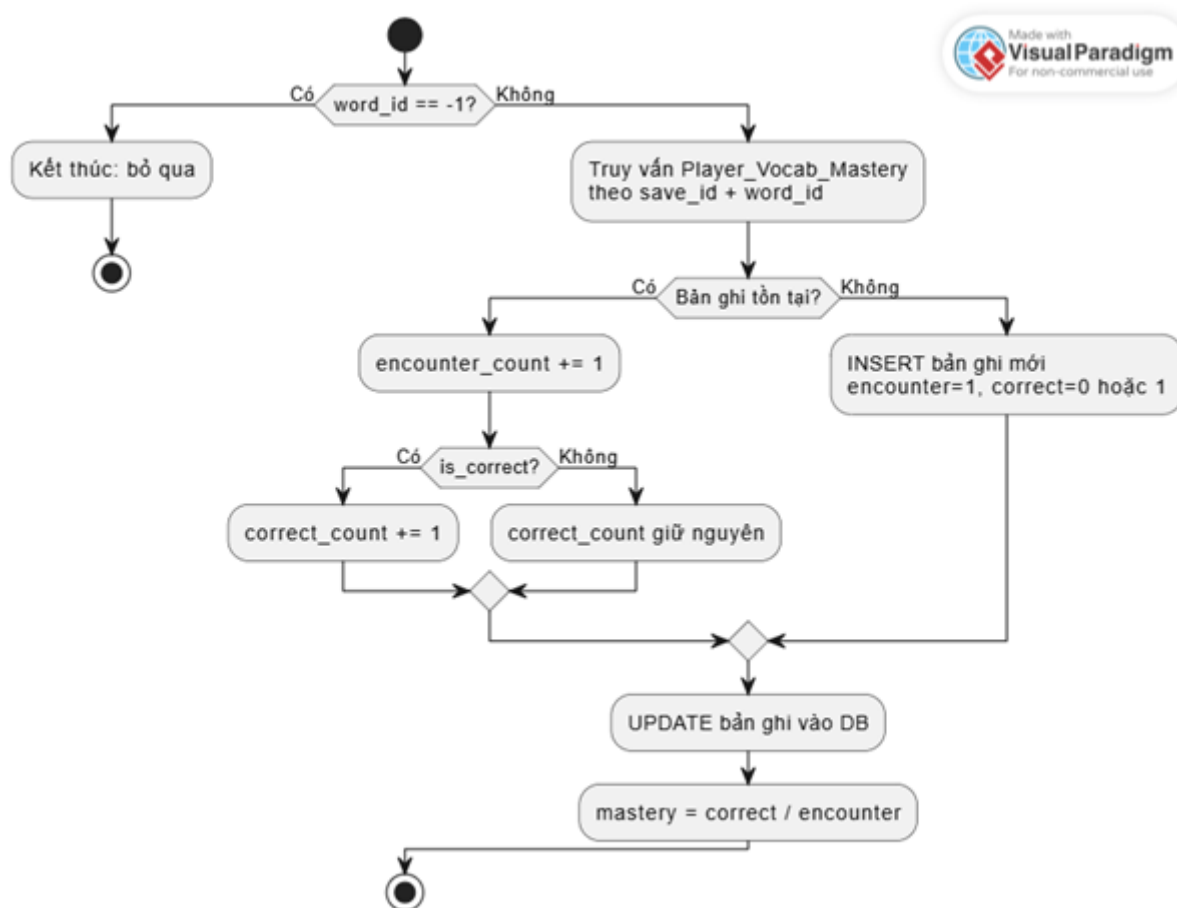
29
30 FUNCTION get_mastery(save_id, word_id):
31     record = DB.query(...)
32     IF NOT record EXISTS:
33         RETURN 0.0 -- tu chưa gặp -> mastery mac dinh = 0
34     RETURN record.correct_count / (record.encounter_count + 1)

```

Complexity:

Thao tác	Time	Space	Ghi chú
update_after_combat()	$O(1)$	$O(1)$	UPSERT trên PK kép (save_id, word_id)
get_mastery()	$O(1)$	$O(1)$	Point lookup trên indexed PK
Tính mastery_score	$O(1)$	$O(1)$	Phép chia đơn giản

Activity Diagram - update_after_combat():



6.3.2 Algorithm 2 — DDA Pool Sampling (chọn từ yếu nhất)

Ý tưởng: Bài toán chọn từ để hỏi cần cân bằng hai mục tiêu xung đột: luôn ưu tiên từ yếu nhất (để học hiệu quả) nhưng không bị kẹt lặp lại mãi một từ (UX tệ). Giải pháp 2 bước: SQL lọc ra pool 15 từ yếu nhất trong tier (đảm bảo 100% chọn từ nhóm yếu), sau đó GDScript chọn ngẫu nhiên 1 trong pool đó (đảm bảo không bị kẹt). RANDOM() làm tiebreaker trong SQL xử lý trường hợp tất cả mastery bằng 0 khi người chơi mới bắt đầu.

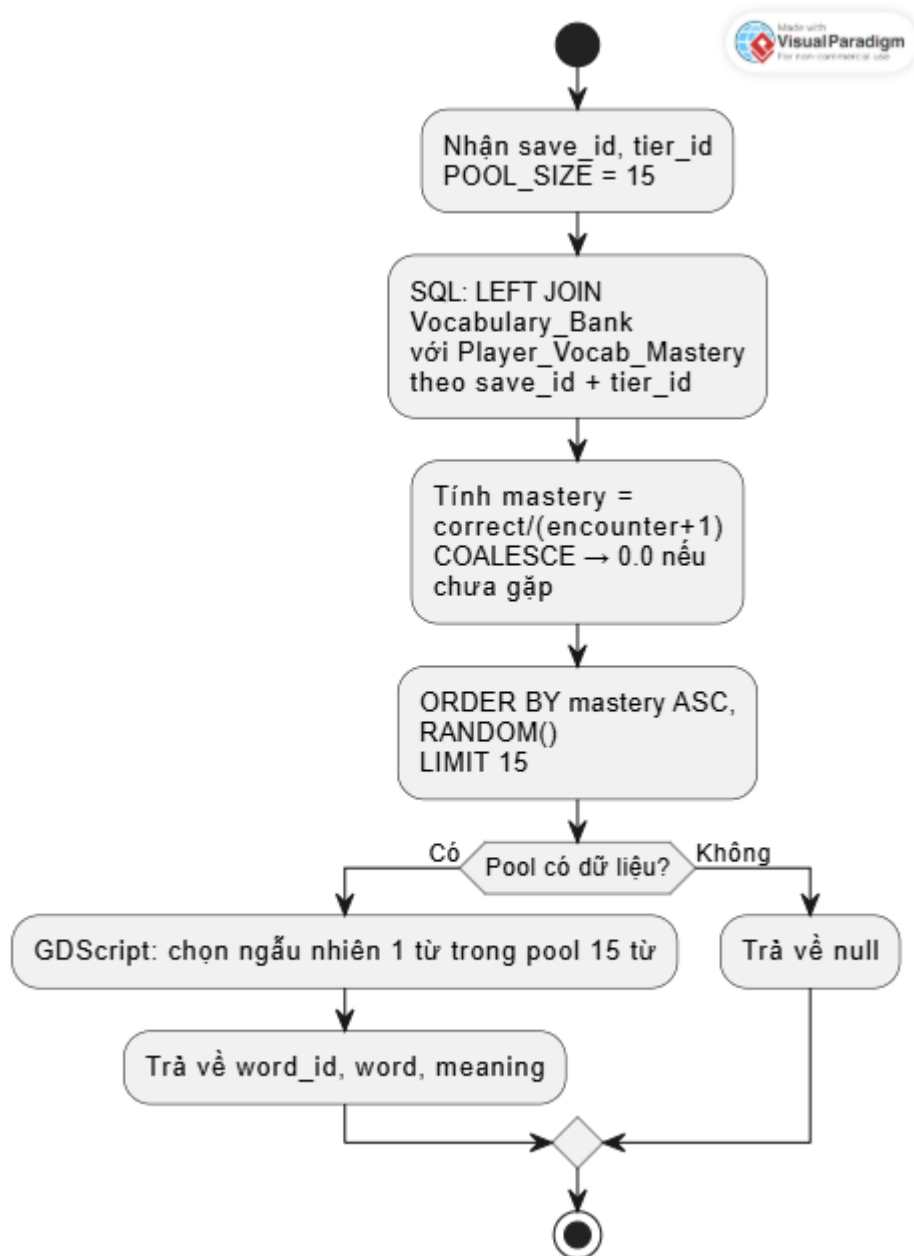
Pseudocode:

```
1 FUNCTION get_weakest_vocab(save_id, tier_id):
2     POOL_SIZE = 15
3
4     -- Buoc 1 (SQL): lay pool tu yeu nhat trong tier
5     pool = DB.query("""
6         SELECT v.word_id, v.word, v.meaning,
7             COALESCE(m.correct_count * 1.0 / (m.encounter_count + 1),
8             0.0)
9             AS mastery_score
10        FROM    Vocabulary_Bank v
11       LEFT JOIN Player_Vocab_Mastery m
12          ON v.word_id = m.word_id AND m.save_id = ?
13       WHERE   v.tier_id = ?
14       ORDER BY mastery_score ASC, RANDOM()
15       LIMIT ?
16     """, [save_id, tier_id, POOL_SIZE])
17
18     IF pool IS EMPTY:
19         RETURN null
20
21     -- Buoc 2 (GDScript): chon ngau nhien trong pool
22     chosen = pool[ RANDOM_INT(0, pool.size() - 1) ]
23     RETURN chosen -- { word_id, word, meaning, mastery_score }
```

Complexity

Thao tác	Time	Space	Ghi chú
SQL query + LEFT JOIN	$O(n \log n)$	$O(n)$	n = số từ trong tier; SQLite sort trên indexed <code>tier_id + mastery_score</code>
LIMIT 15 cắt kết quả	$O(1)$	$O(1)$	Pool cố định 15 phần tử
Random pick trong pool	$O(1)$	$O(1)$	Index ngẫu nhiên vào array 15 phần tử
Tổng thực tế	$O(n \log n)$	$O(15) \approx O(1)$	$n \leq \sim 50$ từ/tier $\rightarrow$ negligible

Activity Diagram - get_weakest_vocab():



6.3.3 Algorithm 3 — DDA Adaptive Prompt (điều chỉnh độ khó)

Ý tưởng: Sau khi chọn được từ mục tiêu, hệ thống cần quyết định đặt câu hỏi ở mức khó nào phù hợp với trạng thái học hiện tại. Rule-based heuristic trên `mastery_score` và `avg_mastery` toàn tier đủ nhẹ cho local game và cho kết quả tức thì — không cần ML model. `tier_is_advanced` là biến ngưỡng giúp toàn bộ tier nâng bậc khi người chơi đã quen đủ từ vựng chung.

Pseudocode:

```

1 FUNCTION resolve_difficulty_instruction(word_id, save_id, tier_id):
2     record      = get_vocab_record(save_id, word_id)
3     encounter    = record.encounter_count IF record EXISTS ELSE 0
4     mastery      = record.mastery_score   IF record EXISTS ELSE 0.0
5     avg_mastery  = get_tier_avg_mastery(save_id, tier_id)
6
7     THRESHOLD_HARD = 0.4
8     tier_is_advanced = (avg_mastery >= THRESHOLD_HARD)
9

```

```

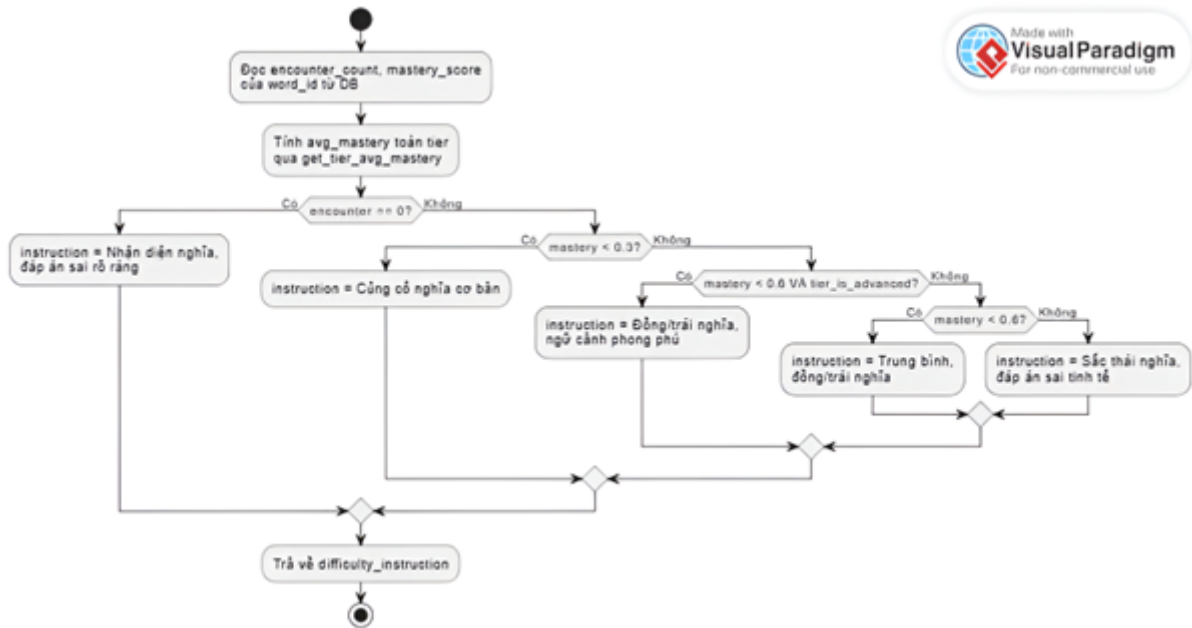
10     IF encounter == 0:
11         RETURN "Lan dau gap: cau hoi nhan dien nghia truc tiep, dap an sai
           ro rang"
12
13     IF mastery < 0.3:
14         RETURN "Hay sai: cung co nghia co ban, khong tang do kho"
15
16     IF mastery < 0.6 AND tier_is_advanced:
17         RETURN "Dang tien bo, tier kha: dong/trai nghia, dien vao cho
           trong nguoc canh phong phu"
18
19     IF mastery < 0.6:
20         RETURN "Dang tien bo: cau hoi muc trung binh, dong/trai nghia"
21
22     RETURN "Thanh thao: sac thai nghia (nuance), dap an sai tinh te kho
           loai tru"
23
24
25 FUNCTION get_tier_avg_mastery(save_id, tier_id):
26     result = DB.query("""
27         SELECT AVG(COALESCE(m.correct_count * 1.0 / (m.encounter_count +
           1), 0.0))
28         FROM     Vocabulary_Bank v
29         LEFT JOIN Player_Vocab_Mastery m
30                 ON v.word_id = m.word_id AND m.save_id = ?
31         WHERE    v.tier_id = ?
32     """)
33     RETURN result -- float

```

Complexity:

Thao tác	Time	Space	Ghi chú
get_tier_avg_mastery()	$O(n)$	$O(1)$	Quét toàn bộ n từ trong tier để tính AVG
Rule matching	$O(1)$	$O(1)$	Chuỗi if-else cố định, không phụ thuộc n
Tổng	$O(n)$	$O(1)$	$n \leq \sim 50$ từ/tier

Activity Diagram - resolve_difficulty_instruction():



6.3.4 Algorithm 4 — Asynchronous Pre-generation Engine

Ý tưởng: Producer-Consumer pattern với multi-tier queue. **AIManager** đóng vai producer chạy ngầm, liên tục nạp câu hỏi vào `question_queues[tier_id]` trước khi người chơi cần. **BattleScene** đóng vai consumer, lấy tức thì từ RAM — không cần chờ AI. Flag `is_generating` đảm bảo Ollama không nhận quá 1 request song song. Priority scheduling ưu tiên tier hiện tại. Fallback `word_id = -1` và auto-retry 2 giây xử lý mọi lỗi runtime mà không crash game.

Pseudocode:

```

1  -- Cấu trúc dữ liệu
2  question_queues = { 1: [], 2: [] }    -- Dictionary of Arrays
3  MANAGED_TIERS   = [1, 2]
4  MAX_QUEUE_SIZE  = 5
5  is_generating   = false
6  current_tier_id = 1
7
8
9  FUNCTION get_question():
10     queue = question_queues[current_tier_id]
11     IF queue IS EMPTY:
12         check_and_fill_all_queues()    -- kích hoạt nạp khan cấp
13         RETURN build_fallback_question() -- word_id = -1, trả ngay
14
15     question = queue.pop_front()

```

```

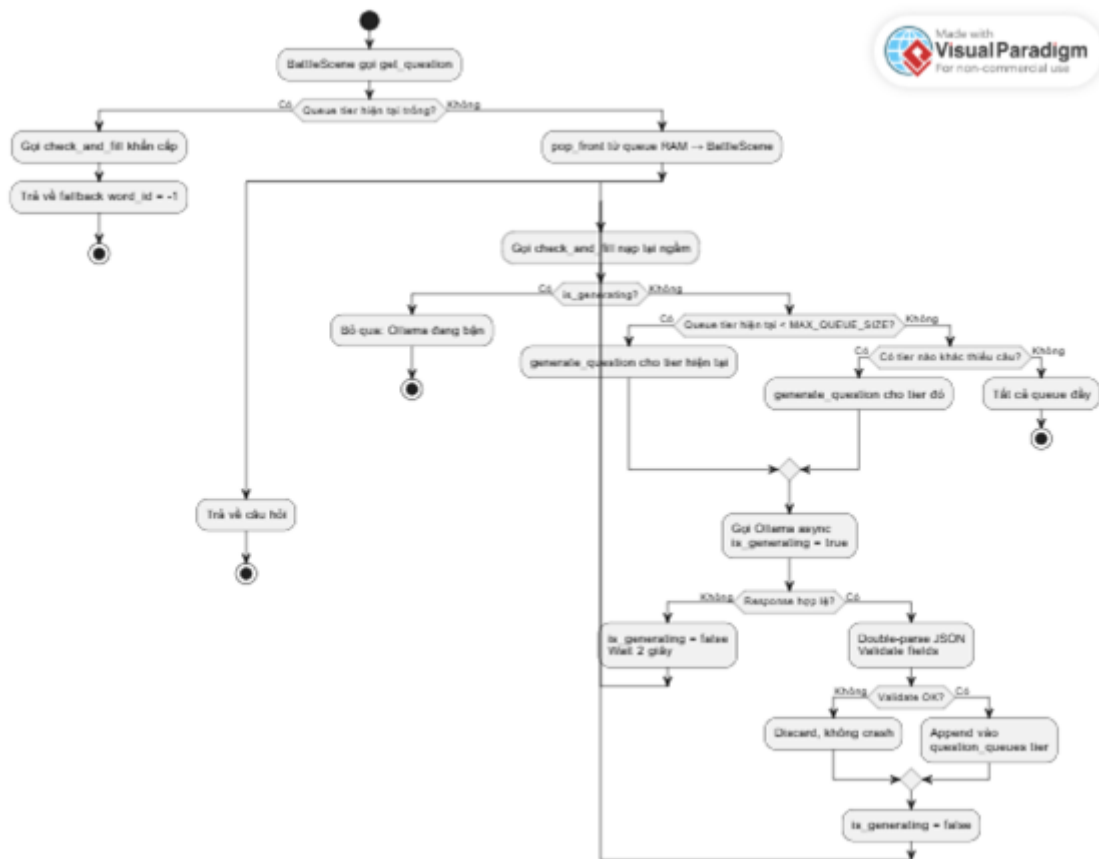
16     check_and_fill_all_queues() -- nap lai ngam sau khi rut
17     RETURN question
18
19
20 FUNCTION check_and_fill_all_queues():
21     IF is_generating:
22         RETURN -- Ollama dang ban, khong gui them
23
24     -- Uu tien 1: nap tier hien tai nguoi choi dang dung
25     IF question_queues[current_tier_id].size() < MAX_QUEUE_SIZE:
26         generate_question_for_tier(current_tier_id)
27     RETURN
28
29     -- Uu tien 2: nap ngam cac tier con thieu
30     FOR tier IN MANAGED_TIERS:
31         IF question_queues[tier].size() < MAX_QUEUE_SIZE:
32             generate_question_for_tier(tier)
33     RETURN
34
35
36 FUNCTION generate_question_for_tier(tier_id):
37     is_generating = true
38     vocab = DatabaseManager.get_weakest_vocab(save_id, tier_id)
39     prompt = build_prompt(vocab, resolve_difficulty_instruction(...))
40
41     response = await Ollama.request(prompt) -- async HTTP
42
43     IF response.code != 200 OR JSON invalid:
44         is_generating = false
45         wait(2 seconds)
46         check_and_fill_all_queues() -- auto-retry
47     RETURN
48
49     question = parse_response(response) -- double-parse
50     IF question IS VALID:
51         question_queues[tier_id].append(question)
52
53     is_generating = false
54     check_and_fill_all_queues() -- nap tiep nhe neu con thieu
55
56
57 FUNCTION build_fallback_question():
58     RETURN {
59         word_id: -1,
60         question: "What does 'Forest' mean?",
61         A: "Rung", B: "Bien", C: "Nui", D: "Dong",
62         correct_answer: "A",
63         explanation: "Forest = Rung (cau du phong)"
64     }

```

Complexity:

Thao tác	Time	Space	Ghi chú
get_question() từ RAM	$O(1)$	$O(1)$	Pop từ đầu array
check_and_fill_all_queues()	$O(T)$	$O(1)$	T = số tier ($T = 2$, hằng số)
Queue RAM tổng	—	$O(T \times K)$	2 tier $\times$ 5 câu = 10 objects $\rightarrow O(1)$
Latency khi tier switch	$O(1)$	$O(1)$	Chỉ cập nhật <code>current_tier_id</code>
AI generation (Ollama)	$O(1)^*$	$O(1)$	*3–10 giây wall-clock chạy bất đồng bộ

Activity Diagram - get_question() + check_and_fill_all_queues():



PlantUML Activity Diagram — get_question() + check_and_fill_all_queues():

```

1 @startuml
2 start
3 :BattleScene gọi get_question;
4 if (Queue tier hiện tại trống?) then (Có)
5     :Gọi check_and_fill_khan_cap;
6     :Trả về fallback word_id = -1;
7     stop
8 else (Không)
9     :pop_front từ queue RAM -> BattleScene;
10    split
11        :Trả về câu hỏi;
12        stop
13    split again
14        label quay_lai_nạp_ngam
15        :Gọi check_and_fill nạp lại ngam;

```

```

16     if (is_generating?) then (Co)
17         :Bo qua: Ollama dang ban;
18         stop
19     else (Khong)
20         if (Queue tier hien tai < MAX_QUEUE_SIZE?) then (Co)
21             :generate_question cho tier hien tai;
22         else (Khong)
23             if (Co tier nao khac thieu cau?) then (Co)
24                 :generate_question cho tier do;
25             else (Khong)
26                 :Tat ca queue day;
27                 stop
28             endif
29         endif
30         :Goi Ollama async\nis_generating = true;
31         if (Response hop le?) then (Khong)
32             :is_generating = false\nWait 2 giay;
33             goto quay_lai_nap_ngam
34         else (Co)
35             :Double-parse JSON\nValidate fields;
36             if (Validate OK?) then (Khong)
37                 :Discard, khong crash;
38             else (Co)
39                 :Append vao\nquestion_queues tier;
40             endif
41             :is_generating = false;
42             goto quay_lai_nap_ngam
43         endif
44     endif
45 end split
46 endif
47 @enduml

```

6.3.5 Algorithm 5 — Structured Prompt Engineering & Double-parse

Ý tưởng: Qwen2.5-3B chạy local không ổn định bằng cloud model. Chiến lược phòng thủ 2 lớp: lớp 1 ràng buộc output tại cấp model (tham số `format: json` của Ollama API) và cấp prompt (khai báo schema tường minh, cấm text ngoài JSON); lớp 2 là pipeline parse-validate-discard trong code để không bao giờ crash dù AI trả output hỏng. `word_id = -1` là sentinel value cho phép toàn bộ pipeline nhận biết câu fallback mà không cần điều kiện đặc biệt ở mỗi bước.

Pseudocode:

```

1 FUNCTION build_prompt(vocab, difficulty_instruction):
2     RETURN """
3         [ROLE]
4         Ban la Elaria, he thong tao thu thach tieng Anh cho game RPG rung.
5
6         [TARGET WORD]
7         Tu: {vocab.word}
8         Nghĩa: {vocab.meaning}
9
10        [DIFFICULTY INSTRUCTION]
11        {difficulty_instruction}

```

```

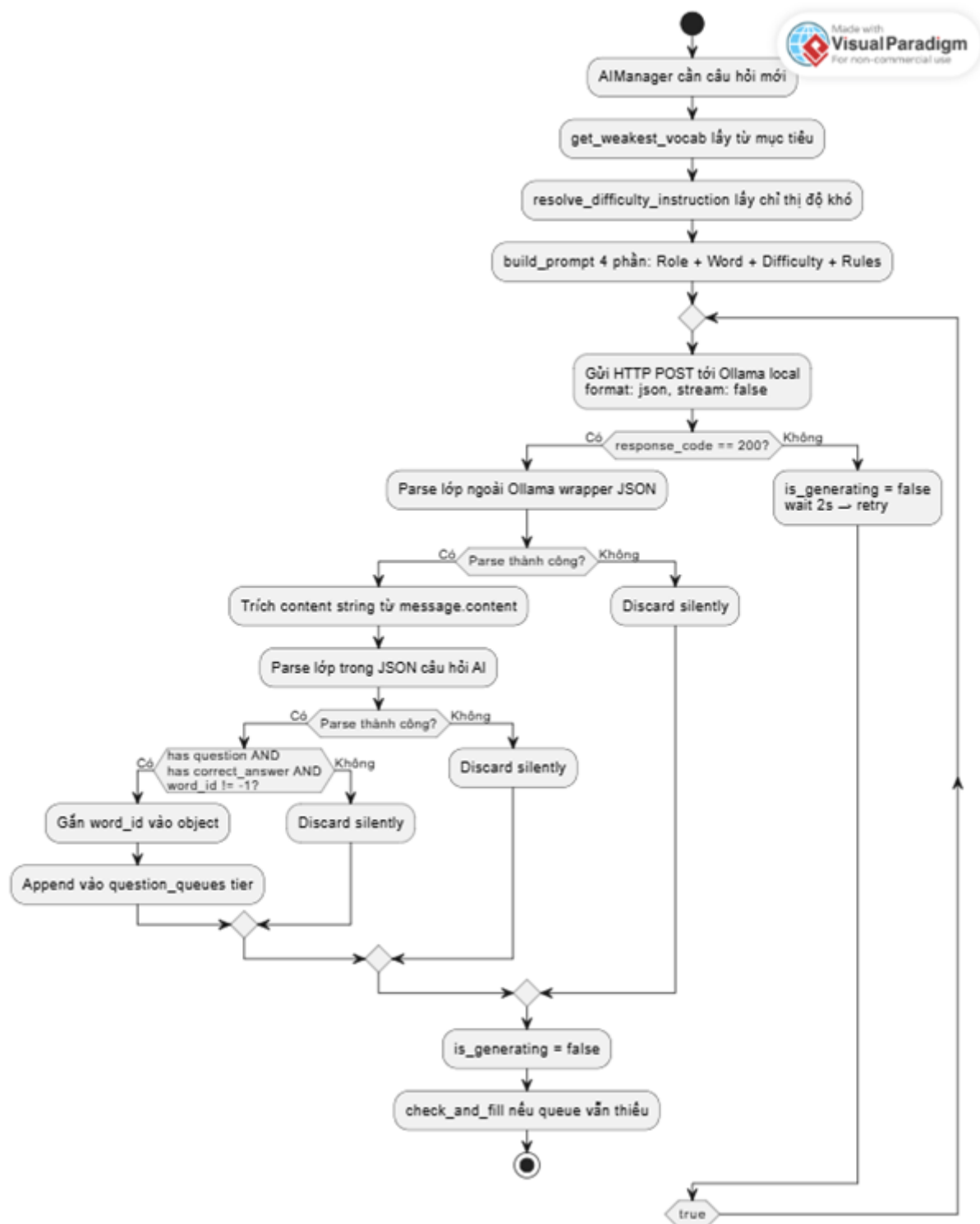
12
13 [OUTPUT RULES]
14 Tra ve JSON hop le voi dung cac truong sau, khong them text nao khac:
15 {
16     "question":      "<cau hoi trac nghiem>",
17     "A":             "<dap an A>",
18     "B":             "<dap an B>",
19     "C":             "<dap an C>",
20     "D":             "<dap an D>",
21     "correct_answer": "<A hoac B hoac C hoac D>",
22     "explanation":     "<giai thich ngan tai sao dap an dung>"
23 }
24 ""
25
26
27 FUNCTION parse_and_validate(raw_response, word_id):
28     TRY:
29         -- Lop 1: parse Ollama wrapper
30         outer = JSON.parse(raw_response)
31         content_str = outer["message"]["content"]
32
33         -- Lop 2: parse JSON cau hoi do AI sinh
34         q_data = JSON.parse(content_str)
35
36         -- Validate bat buoc
37         IF NOT q_data.has("question"): RETURN null
38         IF NOT q_data.has("correct_answer"): RETURN null
39         IF word_id == -1: RETURN null
40
41         q_data["word_id"] = word_id
42         RETURN q_data
43
44     CATCH any error:
45         RETURN null -- discard silently, khong crash
46
47
48 FUNCTION _on_ollama_response(response):
49     IF response.code != 200:
50         handle_error()
51     RETURN
52
53     question = parse_and_validate(response.body, current_word_id)
54     IF question IS NOT null:
55         question_queues[current_tier_id].append(question)

```

Complexity:

Thao tác	Time	Space	Ghi chú
build_prompt()	$O(1)$	$O(L)$	L = độ dài prompt, hằng số
JSON.parse() lần 1	$O(R)$	$O(R)$	R = kích thước response (~ 500 chars)
JSON.parse() lần 2	$O(C)$	$O(C)$	C = kích thước content (~ 300 chars)
Validate (has-check)	$O(1)$	$O(1)$	2 key lookup cố định
Tổng	$O(R)$	$O(R)$	R nhỏ, thực tế negligible

Activity Diagram - build → request → double-parse → validate



Tóm tắt complexity của 5 thuật toán cốt lõi:

#	Thuật toán	Time	Space
1	Knowledge Tracing (mastery score)	$O(1)$	$O(1)$
2	DDA Pool Sampling	$O(n \log n)$	$O(1)$
3	DDA Adaptive Prompt	$O(n)$	$O(1)$
4	Async Pre-generation Engine	$O(1)$ runtime / $O(T \times K)$ build	$O(T \times K)$
5	Structured Prompt + Double-parse	$O(R)$	$O(R)$

Bảng 5: *

Với $n \leq 50$ từ/tier, $T = 2$ tier, $K = 5$ câu/queue, $R \approx 500$ ký tự — tất cả đều negligible về mặt thực tế.

6.4 Optimization Strategy

Phần này trình bày các chiến lược tối ưu hóa được áp dụng cho từng tầng của hệ thống, từ cơ sở dữ liệu, thuật toán lõi, đến pipeline sinh câu hỏi AI. Mỗi chiến lược đều xuất phát từ một vấn đề thực tế quan sát được trong quá trình phát triển, không phải tối ưu hóa lý thuyết thuần túy.

6.4.1 Phần 1 — Cơ sở dữ liệu (SQLite)

Vấn đề: Hàm `get_weakest_vocab()` được gọi trước mỗi lần sinh câu hỏi. Nếu query chạy chậm, toàn bộ pipeline AI bị trì hoãn theo.

Chiến lược áp dụng:

- **Composite Index trên (save_id, tier_id):** Query cốt lõi của DDA lọc đồng thời theo hai cột `save_id` và `tier_id`. Không có index, SQLite phải quét toàn bộ bảng `Player_Vocab_Mastery` mỗi lần gọi. Composite index cho phép SQLite nhảy thẳng đến đúng partition dữ liệu:

```
1 CREATE INDEX IF NOT EXISTS idx_mastery_save_tier
2 ON Player_Vocab_Mastery(save_id, tier_id);
```

Với dataset nhỏ (~ 50 từ/tier), lợi ích chưa rõ ràng ngay, nhưng khi mở rộng lên hàng trăm từ ở các chapter sau, index này ngăn hệ số tăng trưởng query time từ $O(n)$ về $O(\log n)$.

- **COALESCE thay vì subquery:** Từ chưa gặp lần nào không có bản ghi trong `Player_Vocab_Mastery`. Thay vì dùng subquery kiểm tra sự tồn tại (chạy hai lần), `LEFT JOIN` kết hợp `COALESCE` xử lý trong một lần duy nhất:

```
1 -- Không tối ưu: 2 lần truy cập DB
2 IF NOT EXISTS (SELECT 1 FROM mastery WHERE word_id = ?) THEN mastery = 0.0
3
4 -- Tối ưu: 1 lần, NULL -> 0.0 inline
5 COALESCE(m.correct_count * 1.0 / (m.encounter_count + 1), 0.0) AS
   mastery_score
```

- **INSERT OR IGNORE cho vocabulary seeding:** Hàm `_seed_from_csv()` chạy mỗi lần khởi động game. Thay vì kiểm tra trước rồi mới insert (2 lần truy cập), `INSERT OR IGNORE` kết hợp với `UNIQUE(word)` constraint xử lý idempotency trong một lệnh duy nhất — đảm bảo chạy nhiều lần không sinh dữ liệu trùng, không cần thêm logic kiểm tra.

6.4.2 Phần 2 — Thuật toán DDA

Vấn đề ban đầu: ORDER BY RANDOM() thuần có thể chọn từ đã thành thạo nếu may mắn. Khi tất cả từ trong tier có mastery = 0.0 (người chơi mới), ORDER BY mastery ASC không phân biệt được các từ với nhau, dẫn đến hệ thống kẹt lặp lại word_id 1–5 liên tục.

Chiến lược áp dụng:

- Pool Sampling 2 bước thay vì random thuần:

Cách tiếp cận	Vấn đề
ORDER BY RANDOM()	Có thể chọn từ thành thạo
ORDER BY mastery ASC (chỉ lấy 1)	Kẹt word_id 1–5 khi mastery bằng nhau
Pool 15 + RANDOM() tiebreaker	Đảm bảo từ yếu + không kẹt

Giải pháp cuối kết hợp được cả hai mục tiêu mà không tăng complexity đáng kể: SQL vẫn chỉ chạy một lần, GDScript chỉ thêm một phép random trên array 15 phần tử.

- pool_size = 15 thay vì 5: Tăng từ 5 lên 15 giải quyết trường hợp tier có nhiều từ cùng mastery thấp — pool lớn hơn giúp phân phối đều hơn, người chơi không cảm thấy bị hỏi lặp đi lặp lại cùng một từ trong nhiều trận liên tiếp.
- tier_is_advanced như một ngưỡng toàn cục: Thay vì đánh giá độ khó độc lập cho từng từ, biến tier_is_advanced = (avg_mastery ≥ 0.4) cho phép toàn bộ tier "lên hạng" khi người chơi đã quen đủ từ vựng. Điều này giúp tránh tình huống một từ lẻ bị đẩy lên độ khó cao trong khi người chơi vẫn đang vật lộn với phần lớn tier đó.

6.4.3 Phần 3 — Async Pre-generation Engine

Vấn đề: Ollama chạy local tốn 3–10 giây mỗi request. Sinh câu hỏi theo yêu cầu (on-demand) buộc người chơi chờ trước mỗi trận.

Chiến lược áp dụng:

- Pre-generation — sinh câu hỏi trước khi cần: Câu hỏi được tạo sẵn trong lúc người chơi di chuyển trên bản đồ (thời gian rảnh của hệ thống), lưu vào RAM queue. Khi vào trận, get_question() chỉ cần pop_front() — latency bằng 0 từ góc nhìn người chơi.
- Multi-tier Queue thay vì queue chung: Queue chung có nhược điểm cơ bản: khi người chơi đổi tier, toàn bộ queue bị xóa và phải sinh lại từ đầu, gây ra 3–10 giây chờ. Dictionary of Arrays giải quyết triệt để:

```
var question_queues: Dictionary = { 1: [], 2: [] }
```

Mỗi tier giữ queue riêng, việc đổi tier chỉ cập nhật con trỏ current_tier_id — không đụng đến dữ liệu đã cache.

- Priority Scheduling — ưu tiên tier hiện tại: check_and_fill_all_queues() không nạp tất cả tier song song (Ollama chỉ xử lý 1 request tại 1 thời điểm). Thay vào đó, hàm ưu tiên tier đang dùng trước, rồi mới nạp ngầm các tier còn lại — đảm bảo queue của tier hiện tại không bao giờ trống khi người chơi cần.

- **Flag `is_generating` ngăn request song song:** Ollama single-threaded không xử lý được concurrent request. Flag boolean đơn giản đủ để serialize toàn bộ pipeline:

```
1 if is_generating:
2     return -- bo qua, không gui thêm request
```

Chi phí: 1 biến boolean. Lợi ích: không bao giờ gây quá tải Ollama.

- **Fallback + Auto-retry không cần can thiệp thủ công:** Khi AI lỗi, hệ thống không crash mà trả về câu dự phòng (`word_id = -1`) để game tiếp tục. Sau 2 giây, `check_and_fill_all_queues()` tự gọi lại — người chơi không nhận ra bất kỳ sự gián đoạn nào.

6.4.4 Phần 4 — Prompt Engineering

Vấn đề: Qwen2.5-3B chạy local không ổn định — dễ thêm text ngoài JSON, bọc output trong markdown fence, hoặc bỏ sót field bắt buộc.

Chiến lược áp dụng:

- **Ràng buộc 2 lớp:** Lớp 1 tại cấp model: tham số `format: json` của Ollama API buộc model enforce JSON ở cấp tokenizer — xác suất sinh text tự do giảm đáng kể. Lớp 2 tại cấp prompt: khai báo tường minh JSON schema và cấm thêm bất kỳ text nào khác, tạo neo ngữ nghĩa cho model.
- **`temperature = 0.45` — cân bằng đa dạng và ổn định:** Giá trị này được chọn sau quá trình thử nghiệm thực tế:

Temperature	Hệ quả
< 0.3	Câu hỏi lặp lại, đáp án sai quá giống nhau
0.45	Đủ đa dạng câu hỏi, đủ ổn định để đúng JSON
> 0.7	Format JSON không ổn định, field hay bị thiếu

- **`stream: false` — đơn giản hóa parsing:** Nhận toàn bộ response một lần thay vì xử lý stream token-by-token. Đánh đổi: người chơi không thấy câu hỏi "hiện dần", nhưng không quan trọng vì câu hỏi được pre-generate ngầm, không hiển thị trong lúc sinh.
- **Double-parse + validate trước khi enqueue:** Pipeline parse-validate-discard đảm bảo chỉ câu hỏi hợp lệ mới vào queue:

Parse lớp ngoài → Parse lớp trong → Kiểm tra 2 field bắt buộc → Enqueue / Discard silently

Discard silently — không throw exception, không crash — giúp hệ thống tự phục hồi mà không cần logic xử lý lỗi phức tạp ở tầng trên.

6.4.5 Tổng kết các đánh đổi (Trade-offs)

Mỗi chiến lược tối ưu hóa đều kèm theo một đánh đổi được chấp nhận có chủ đích:

Chiến lược	Lợi ích	Đánh đổi chấp nhận
Multi-tier Queue	Latency = 0 khi đổi tier	RAM tăng nhẹ ($\sim 2\text{KB}$)
Pool size = 15	Phân phối đều, không kẹt từ	Query trả về nhiều hơn 1 bản ghi
<code>stream: false</code>	Parsing đơn giản	Không có hiệu ứng "hiện dần"
temperature = 0.45	JSON ổn định	Câu hỏi ít sáng tạo hơn mức cao
Discard silently	Không crash khi AI lỗi	Queue có thể thiếu 1 câu tạm thời
Pre-generation ngầm	Người chơi không chờ AI	Câu hỏi sinh sẵn có thể không dùng đến